optimization: the Optimization program — user guide

optimization-ai999x 1.0.3, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Optimization covers linear and integer programming, convex and nonlinear optimisation, assignment, transportation and shortest-route problems, scheduling, inventory, queueing, game theory and multi-criteria decisions. This guide is about `optimization`, the model as a self-contained program for Windows and Linux: 72 capabilities in 16 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/optimization-ai999x/); the Python package (`pip install optimization-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.4.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS; the newest version is always under https://downloads.ai999x.com/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Optimization	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-optimization-1.4.0-windows-x86_64.zip
Desktop app: Optimization	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-optimization-1.4.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-windows-x86_64.zip
Command line	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz

Desktop app

Optimization in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-optimization-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-optimization.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-optimization-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-optimization-1.4.0-linux-x86_64.tar.gz
./ai999x-optimization-1.4.0-linux-x86_64/ai999x-optimization
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.4.0-linux-x86_64.tar.gz
./ai999x-1.4.0-linux-x86_64/ai999x
```

Command line — Linux

The bundle holds one program per model; `optimization` is this one. The checksum line should print OK.

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.4.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.4.0-linux-x86_64:$PATH"' >> ~/.bashrc && source
~/.bashrc
optimization --version
```

Command line — Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.4.0-windows-x86_64.zip"
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/$F" -OutFile $F
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F
SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.4.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User")
+ ";$D", "User")
optimization --version
```

In your browser

Open https://www.ai999x.com/models/optimization-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install optimization-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ optimization --version
optimization-ai999x 1.0.3 (Rust build, ax-engine 0.1.0)
```

2.2 Ask in words

Write the question as you would say it: the program reads the quantities and their units, picks the capability and answers.

```
$ optimization ask "Solve the transportation problem with supply [20, 30], demand [10, 25, 15] and
cost matrix [[2, 3, 1], [5, 4, 8]]."
Read as: Transportation problem (minimum shipping cost)
min_cost = 150
shipments = [[5.0,0.0,15.0],[5.0,25.0,0.0]]
plan_unique = yes
dual = [-3.0,0.0,5.0,4.0,4.0]
  by: Transportation problem (minimum shipping cost)  (the transportation LP solved exactly
(rational simplex), certified by its dual (u_i + v_j <= c_ij))
  assumes: total supply >= total demand (surplus stays at the sources at no cost); at most 60
routes
```

2.3 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ optimization search "economic order quantity (eoq)"
eoq                                 Economic order quantity (EOQ)  (inventory management)
eoq_quantity_discount               Order quantity with quantity discounts (all-units)  (inventory
management)
epq_production_lot                  Economic production quantity (EPQ)  (inventory management)
eoq_planned_shortages               EOQ with planned backorders  (inventory management)
newsvendor_normal                   Newsvendor order quantity (normal demand)  (inventory
management / single-period)
newsvendor_discrete                 Newsvendor order quantity (discrete demand)  (inventory
management / single-period)
newsvendor_uniform                  Newsvendor order quantity (uniform demand)  (inventory
management / single-period)
order_up_to_level                   Periodic-review order-up-to level  (inventory management)
… 12 more
```

2.4 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ optimization list
ahp_consistency_ratio                   AHP consistency ratio
ahp_geometric_mean                      AHP weights by the geometric mean method
ahp_priority_vector                     AHP priority vector (principal eigenvector)
all_pairs_shortest_paths                All-pairs shortest distances (Floyd-Warshall)
assignment_max_profit                   Assignment problem (Hungarian method), maximum total value
assignment_min_cost                     Assignment problem (Hungarian method), minimum total cost
bellman_ford_shortest_path              Shortest path with negative weights (Bellman-Ford)
bin_packing                             Bin packing (minimum number of bins)
… 64 more
```

2.5 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ optimization info eoq
Economic order quantity (EOQ)
  minimise D S / Q + H Q / 2 (at the optimum ordering cost equals holding cost)
  in   D        demand per period (units, e.g. per year) [1]
  in   S        fixed cost per order [1]
  in   H        holding cost per unit per period [1]
  out  Q        economic order quantity [1]
  out  total_cost minimum ordering + holding cost per period [1]
  out  orders_per_period number of orders per period [1]
  out  cycle_time time between orders (in periods) [1]
```

2.6 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ optimization solve eoq D=1200 S=50 H=3
Q = 200
total_cost = 600
orders_per_period = 6
cycle_time = 0.1666666667
  by: Economic order quantity (EOQ)  (minimise D S / Q + H Q / 2 (at the optimum ordering cost
equals holding cost))
  D = 1200
  S = 50
  H = 3
  assumes: D, S, H > 0; constant demand, instant replenishment, no shortages
```

2.7 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ optimization solve eoq D=1200 S=50 H=3 --json
{"outcome":"SOLVED","message":"","value":200.0,"unit":"","solved_for":"Q","si":[["D",1200.0],
["S",50.0],["H",3.0]],"values":[["Q",200.0,""],["total_cost",600.0,""],
["orders_per_period",6.0,""],["cycle_time",0.16666666666666666,""]],"target":"Q","phrase":"economic
order quantity","assumptions":["D, S, H > 0","constant demand, instant replenishment, no
shortages"],"name":"Economic order quantity (EOQ)","equation":"minimise D S / Q + H Q / 2 (at the
optimum ordering cost equals holding cost)"}
```

2.8 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ optimization solve eoq D=1200 S=50
NEEDS_INPUT: eoq needs H (holding cost per unit per period). Example: solve('eoq', D=1200, S=50,
H=3).
```

2.9 Draw the figures

The live figures for your own inputs, written as SVG files into the folder given with --out (here the current folder); open them in any browser. Without --out the SVG text comes back as JSON.

```
$ optimization figures eoq D=1200 S=50 H=3 --out .
[
  {
    "slug": "eoq-live-1",
    "title": "Economic order quantity as holding cost per unit per period varies",
    "caption": "Economic order quantity as H runs from 0.75 to 5.25, every other input as you
entered it (41 solutions). Marked: your H = 3 gives Q = 200. Your inputs: D = 1200, S = 50, H =
3.",
    "file": ".\\eoq-live-1.svg"
  },
  {
    "slug": "eoq-live-2",
    "title": "Economic order quantity as fixed cost per order varies",
    "caption": "Economic order quantity as S runs from 12.5 to 87.5, every other input as you
entered it (29 solutions). Marked: your S = 50 gives Q = 200. Your inputs: D = 1200, S = 50, H =
3.",
    "file": ".\\eoq-live-2.svg"
  }
]
```

2.10 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | optimization batch

`cases.jsonl`:

```
{"key": "eoq", "inputs": {"D": 1200, "S": 50, "H": 3}}
{"key": "ahp_consistency_ratio", "inputs": {"judgments": "[3, 5, 2]"}}
```

```
$ optimization batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":200.0,"unit":"","solved_for":"Q","si":[["D",1200.0],
["S",50.0],["H",3.0]],"values":[["Q",200.0,""],["total_cost",600.0,""],
["orders_per_period",6.0,""],["cycle_time",0.16666666666666666,""]],"target":"Q","phrase":"economic
order quantity","assumptions":["D, S, H > 0","constant demand, instant replenishment, no
shortages"],"name":"Economic order quantity (EOQ)","equation":"minimise D S / Q + H Q / 2 (at the
optimum ordering cost equals holding cost)"}
{"outcome":"SOLVED","message":"","value":0.0031849983307239735,"unit":"","solved_for":"CR","values":
[["CR",0.0031849983307239735,""],["CI",0.0018472990318199045,""],
["lambda_max",3.00369459806364,""],["weights",
[0.6483290138222366,0.2296507940626371,0.12202019211512621],""],
["acceptable","yes",""]],"target":"CR","phrase":"consistency ratio","assumptions":["3 to 10
criteria","RI = 0.58, 0.90, 1.12, 1.24, 1.32, 1.41, 1.45, 1.49 for n = 3..10 (Saaty)"],"name":"AHP
consistency ratio","equation":"principal eigenpair by power iteration (certified as an eigenpair),
then CI and CR"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `optimization info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]" or "v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`optimization solve ... --json` prints one answer as one line of JSON. `optimization batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | optimization batch`. `optimization catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["optimization", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`optimization figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

```
$ optimization figures eoq D=1200 S=50 H=3 --out .
[
  {
    "slug": "eoq-live-1",
    "title": "Economic order quantity as holding cost per unit per period varies",
    "caption": "Economic order quantity as H runs from 0.75 to 5.25, every other input as you
entered it (41 solutions). Marked: your H = 3 gives Q = 200. Your inputs: D = 1200, S = 50, H =
3.",
    "file": ".\\eoq-live-1.svg"
  },
  {
    "slug": "eoq-live-2",
    "title": "Economic order quantity as fixed cost per order varies",
    "caption": "Economic order quantity as S runs from 12.5 to 87.5, every other input as you
entered it (29 solutions). Marked: your S = 50 gives Q = 200. Your inputs: D = 1200, S = 50, H =
3.",
    "file": ".\\eoq-live-2.svg"
  }
]
```

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `optimization_ai999x.wasm` (https://www.ai999x.com/compute/rust/optimization_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("optimization_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `optimization info <id>` shows the same inputs and outputs in the terminal.

Calculus optimisation (5)

Absolute extrema of a polynomial on a closed interval — `polynomial_extrema_interval`

The absolute minimum and maximum of a polynomial on [lo, hi] (closed-interval method: endpoints and critical points), and where they are attained.

$$\min_{[a,b]} f = \min\{f(a), f(b), f(c) : f'(c) = 0,\ a < c < b\}$$

symbol		meaning	SI unit
`coefficients`	in	polynomial coefficients, highest power first	—
`lo`	in	left end of the interval	—
`hi`	in	right end of the interval	—
`min_value`	out	absolute minimum	—
`max_value`	out	absolute maximum	—
`argmin_example`	out	a point attaining the minimum (the smallest one on a tie)	—
`argmax_example`	out	a point attaining the maximum (the smallest one on a tie)	—

Applies when: lo < hi; degree 1 to 5.

```
$ optimization solve polynomial_extrema_interval "coefficients=[1, -3, 0, 2]" lo=-1 hi=3
min_value = -2
max_value = 2
argmin_example = -1
argmax_example = 0
  by: Absolute extrema of a polynomial on a closed interval  (real roots of f' in [lo, hi]
(monotone pieces between the roots of f'' bisected to the last float), compared with the endpoints)
  lo = -1
  hi = 3
  assumes: lo < hi; degree 1 to 5
```

What the worked example means: This calculation determines the global extrema of a polynomial on the closed interval from -1 to 3. It evaluates the function at the endpoints and all real critical points where the derivative is zero within the interval. The smallest resulting value is reported as the absolute minimum, -2, attained at -1, while the largest is the absolute maximum, 2, found at 0.

Closed cylinder of least surface area for a given volume — `can_min_surface`

The radius, height and surface area of the closed cylindrical can that holds a volume V with the least material.

$$r^* = \left(\frac{V}{2\pi}\right)^{1/3}, \quad h^* = 2r^*$$

symbol		meaning	SI unit
`V`	in	volume (cube of the length unit)	—
`radius`	out	optimal radius	—
`height`	out	optimal height (equal to the diameter)	—
`min_area`	out	least total surface area	—

Applies when: V > 0; closed top and bottom.

```
$ optimization solve can_min_surface V=500
radius = 4.301270069
height = 8.602540138
min_area = 348.7342055
  by: Closed cylinder of least surface area for a given volume  (A(r) = 2 pi r^2 + 2V/r; A'(r) = 0)
  V = 500
  assumes: V > 0; closed top and bottom
```

What the worked example means: This calculation determines the dimensions for a closed cylindrical container that holds a specific volume while using the least amount of material. Given an input volume of 500, the method finds the optimal radius to be 4.30127 and the corresponding height to be 8.60254. The resulting minimum surface area required is 348.734. This ensures efficient packaging design by balancing the top, bottom, and side surfaces.

Critical point of a two-variable quadratic and its classification — `critical_point_two_variables`

For f(x, y) = a x^2 + b xy + c y^2 + d x + e y + f0: the critical point, its value and whether it is a local minimum, a local maximum or a saddle point (second-derivative test).

$$D = f_{xx} f_{yy} - f_{xy}^2 = 4ac - b^2$$

symbol		meaning	SI unit
`a`	in	coefficient of x^2	—
`b`	in	coefficient of xy	—
`c`	in	coefficient of y^2	—
`d`	in	coefficient of x (optional)	—
`e`	in	coefficient of y (optional)	—
`f0`	in	constant term (optional)	—
`f_value`	out	value of f at the critical point	—
`x`	out	x of the critical point	—
`y`	out	y of the critical point	—
`type`	out	local minimum, local maximum or saddle point	—

Applies when: 4ac - b^2 != 0 (an isolated critical point).

```
$ optimization solve critical_point_two_variables a=3 b=2 c=1 d=-4 e=6 f0=1
f_value = -20.5
x = 2.5
y = -5.5
type = local minimum
  by: Critical point of a two-variable quadratic and its classification  (grad f = 0 solved
exactly; D = f_xx f_yy - f_xy^2 = 4ac - b^2 classifies it)
  a = 3
  b = 2
  c = 1
  d = -4
  e = 6
  f0 = 1
  assumes: 4ac - b^2 != 0 (an isolated critical point)
```

What the worked example means: This calculation identifies the stationary point for the quadratic function defined by coefficients 3, 2, 1, -4, 6, and 1. Solving the gradient equal to zero shows that the critical point occurs at coordinates x equals 2.5 and y equals -5.5. The value of the function at this specific location is -20.5. Based on the second derivative test, which examines the curvature using the determinant of the Hessian matrix, the nature of this point is classified as a local minimum. This classification indicates whether the surface curves upward like a bowl, downward like a dome, or forms a saddle shape at this coordinate.

Largest rectangle for a given length of fencing — `fence_max_area`

The dimensions and maximum area of a rectangle enclosed by a fence of length L, on four sides or on three sides with a wall or river as the fourth.

$$A_{\max} = \frac{L^2}{16}\ (4\text{ sides}), \quad \frac{L^2}{8}\ (3\text{ sides})$$

symbol		meaning	SI unit
`L`	in	total length of fencing (any length unit)	—
`wall`	in	true when one side lies along a wall or river and needs no fence (optional)	—
`width`	out	side perpendicular to the wall (or one side of the square)	—
`length`	out	side parallel to the wall	—
`max_area`	out	maximum enclosed area (square of the length unit)	—

Applies when: L > 0.

```
$ optimization solve fence_max_area wall=False L=200
width = 50
length = 50
max_area = 2500
  by: Largest rectangle for a given length of fencing  (area as a function of one side, derivative
set to zero)
  L = 200
  assumes: L > 0
```

What the worked example means: This calculation finds the largest rectangular area possible for a given fencing length. Using the total fence amount 200, it determines the optimal side dimensions. The resulting

maximum area is 2500, achieved when the perpendicular side is 50 and the parallel side is 50. This ensures the most efficient use of boundary materials.

Open box of maximum volume from a rectangular sheet — `open_box_max_volume`

Squares of side x are cut from the corners of an L by W sheet and the sides folded up: the cut that maximises the volume, and that volume.

$$x^* = \frac{(L+W) - \sqrt{L^2 - LW + W^2}}{6}$$

symbol		meaning	SI unit
`L`	in	sheet length	—
`W`	in	sheet width	—
`cut`	out	side of the corner squares at the maximum	—
`max_volume`	out	maximum volume	—

Applies when: L > 0 and W > 0.

```
$ optimization solve open_box_max_volume L=12 W=12
cut = 2
max_volume = 128
  by: Open box of maximum volume from a rectangular sheet  (V(x) = x (L - 2x)(W - 2x); V'(x) = 0,
the root in (0, min(L, W)/2))
  L = 12
  W = 12
  assumes: L > 0 and W > 0
```

What the worked example means: This calculation determines the side length of squares to cut from the corners of a rectangular sheet with length 12 and width 12 to maximize the volume of the resulting open box. The optimal cut size is found to be 2, which yields a maximum volume of 128. This result assumes negligible material thickness and perfect vertical folding.

Cost-volume-profit analysis (1)

Break-even quantity — `break_even_point`

The number of units at which revenue equals total cost (fixed plus variable), and the revenue there.

$$Q_{BE} = \frac{F}{p - v}$$

symbol		meaning	SI unit
`F_`	in	fixed cost (currency)	—
`p`	in	selling price per unit	—
`v`	in	variable cost per unit	—
`quantity`	out	break-even quantity (units)	—

symbol		meaning	SI unit
`revenue`	out	revenue at break-even	—
`contribution_margin`	out	price minus variable cost per unit	—

Applies when: fixed cost >= 0; price > variable cost.

```
$ optimization solve break_even_point F_=5000 p=25 v=15
quantity = 500
revenue = 12500
contribution_margin = 10
  by: Break-even quantity  (solve p Q = F + v Q)
  F_ = 5000
  p = 25
  v = 15
  assumes: fixed cost >= 0; price > variable cost
```

What the worked example means: This calculation determines the number of units that must be sold to cover all fixed and variable costs, resulting in zero profit. Using a fixed cost of 5000, a selling price of 25, and a variable cost of 15, the model computes a break-even quantity of 500. At this volume, total revenue equals 12500, meaning the business covers its expenses but earns no profit.

Dynamic programming (2)

Discounted Markov decision process (optimal values and policy) — `mdp_discounted`

The optimal expected discounted reward from every state of a finite Markov decision process and an optimal stationary policy.

$$V^*(s) = \max_a \left(R(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, V^*(s') \right)$$

symbol		meaning	SI unit
`R`	in	rewards, rows = states, columns = actions	—
`T`	in	transition probabilities: one row per (state, action) in the order s1a1, s1a2, ..., s2a1, ...; columns = next state	—
`gamma`	in	discount factor (0 < gamma < 1)	—
`value_state_1`	out	optimal value of state 1	—
`values`	out	optimal value of every state	—
`policy`	out	optimal action in each state, 1-based (the lowest-numbered one on a tie)	—

Applies when: 0 < gamma < 1; rows of T sum to 1; at most 10 states and 5 actions.

```
$ optimization solve mdp_discounted "R=[[1, 0], [0, 2]]" "T=[[1, 0], [0, 1], [0, 1], [1, 0]]"
gamma=0.9
value_state_1 = 10
values = [10.0,11.0]
policy = [1.0,2.0]
  by: Discounted Markov decision process (optimal values and policy)  (policy iteration in exact
arithmetic; certified by the Bellman optimality equation)
  gamma = 0.9
  assumes: 0 < gamma < 1; rows of T sum to 1; at most 10 states and 5 actions
```

What the worked example means: This calculation determines the maximum long-term value achievable in each state of a finite Markov decision process. Using policy iteration, it finds the best stationary strategy by maximizing immediate rewards plus discounted future returns. The resulting optimal value for the first state is 10, reflecting the best possible outcome given the input parameters.

Dynamic lot sizing (Wagner-Whitin) — `wagner_whitin_lot_sizing`

The cheapest ordering plan for known demands over several periods with a fixed cost per order and a holding cost per unit carried to the next period (no shortages).

$$C_t = \min_{1 \le s \le t} \left(C_{s-1} + K + h \sum_{u=s}^{t} (u - s)\, d_u \right)$$

symbol		meaning	SI unit
`demand`	in	demand in each period	—
`K`	in	setup (ordering) cost per order	—
`h`	in	holding cost per unit per period	—
`min_cost`	out	minimum total setup plus holding cost	—
`order_quantities_example`	out	one optimal plan: quantity ordered in each period	—
`orders`	out	number of orders in that plan	—

Applies when: demands, K, h >= 0; no initial stock, no shortages, holding charged on end-of-period stock.

```
$ optimization solve wagner_whitin_lot_sizing "demand=[20, 50, 10, 50]" K=100 h=1
min_cost = 270
order_quantities_example = [80.0,0.0,0.0,50.0]
orders = 2
  by: Dynamic lot sizing (Wagner-Whitin)  (Wagner-Whitin dynamic programme (each order covers whole
periods) in exact arithmetic)
  K = 100
  h = 1
  assumes: demands, K, h >= 0; no initial stock, no shortages, holding charged on end-of-period
stock
```

What the worked example means: This calculation determines the least expensive inventory plan by balancing fixed ordering costs against holding costs. Given demands, a setup cost of 100, and a holding cost of 1, the algorithm finds that the optimal strategy incurs a total cost of 270. This result reflects the trade-off where fewer orders reduce setup expenses but increase storage charges for unused stock.

Economic optimisation (2)

Profit-maximising output with linear demand — `profit_max_linear_demand`

With inverse demand p = a - b q and cost C = F + c q, the output, price and profit at the profit maximum.

$$q^* = \frac{a - c}{2b}, \quad p^* = \frac{a + c}{2}$$

symbol		meaning	SI unit
`a`	in	demand intercept (price at zero quantity)	—
`b`	in	demand slope (price drop per unit)	—
`cv`	in	marginal (variable) cost per unit	—
`F_`	in	fixed cost (default 0) (optional)	—
`quantity`	out	profit-maximising quantity	—
`price`	out	price at that quantity	—
`profit`	out	maximum profit	—

Applies when: b > 0; a > c.

```
$ optimization solve profit_max_linear_demand a=100 b=2 F_=300 cv=20
quantity = 20
price = 60
profit = 500
  by: Profit-maximising output with linear demand  (marginal revenue equals marginal cost: a - 2 b
q = c)
  a = 100
  b = 2
  F_ = 300
  cv = 20
  assumes: b > 0; a > c
```

What the worked example means: This calculation determines the optimal output level for a firm with market power facing linear demand. By equating marginal revenue to marginal cost, it finds that the best quantity to produce is 20. At this level, the firm charges a price of 60 to maximize its returns. The resulting maximum profit, after subtracting fixed costs, is 500. This result assumes constant marginal cost and a downward-sloping demand curve where the intercept exceeds the cost.

Revenue-maximising price with linear demand — `revenue_max_linear_demand`

With demand q = alpha - beta p, the price that maximises revenue p q, the quantity sold and the revenue.

$$p^* = \frac{\alpha}{2\beta}, \quad R^* = \frac{\alpha^2}{4\beta}$$

symbol		meaning	SI unit
`alpha`	in	demand at a price of zero	—
`beta`	in	drop in demand per unit increase in price	—

symbol		meaning	SI unit
`price`	out	revenue-maximising price	—
`quantity`	out	quantity sold at that price	—
`revenue`	out	maximum revenue	—

Applies when: alpha > 0; beta > 0.

```
$ optimization solve revenue_max_linear_demand alpha=1000 beta=20
price = 25
quantity = 500
revenue = 12500
  by: Revenue-maximising price with linear demand  (dR/dp = alpha - 2 beta p = 0 (unit elasticity))
  alpha = 1000
  beta = 20
  assumes: alpha > 0; beta > 0
```

What the worked example means: This calculation determines the optimal price to set when demand follows a linear pattern, specifically targeting maximum gross revenue rather than profit. Using the demand intercept of 1000 and the price sensitivity of 20, the model computes that charging 25 will result in selling 500 units. This strategy generates the highest possible income of 12500, assuming that costs are ignored and the market reacts predictably to price changes without competitor interference.

Game theory (4)

2 x 2 zero-sum game (saddle point or mixed-strategy formula) — `zero_sum_2x2`

The value of a 2 x 2 matrix game: the saddle-point value when one exists, otherwise the mixed-strategy solution (probability of each player's first strategy, unique in that case).

$$v = \frac{ad - bc}{a + d - b - c}, \quad p = \frac{d - c}{a + d - b - c}, \quad q = \frac{d - b}{a + d - b - c}$$

symbol		meaning	SI unit
`A`	in	2 x 2 payoff matrix to the row player [[a, b], [c, d]]	—
`value`	out	value of the game	—
`saddle_point`	out	yes when a pure saddle point exists	—
`p_row1`	out	probability the row player plays row 1 (only without a saddle point)	—
`q_col1`	out	probability the column player plays column 1 (only without a saddle point)	—

Applies when: a 2 x 2 matrix.

```
$ optimization solve zero_sum_2x2 "A=[[3, -1], [-2, 4]]"
value = 1
p_row1 = 0.6
q_col1 = 0.5
saddle_point = no
  by: 2 x 2 zero-sum game (saddle point or mixed-strategy formula)  (maximin = minimax gives a
saddle point; otherwise the indifference formulas in exact arithmetic)
  assumes: a 2 x 2 matrix
```

What the worked example means: This calculation determines the equilibrium for a two by two zero sum game using payoff matrix A. If a pure strategy equilibrium exists, indicated by no, the value of the game is 1. When no pure equilibrium exists, the solver computes mixed strategies where the row player selects their first option with probability 0.6 and the column player selects their first option with probability 0.5, ensuring neither can improve their expected outcome unilaterally.

Mixed-strategy Nash equilibrium of a 2 x 2 game — `mixed_nash_2x2`

The equilibrium of a 2 x 2 two-player game without a pure equilibrium: each player's probability of the first strategy (chosen to make the OTHER player indifferent) and the expected payoffs.

$$q = \frac{a_{22} - a_{12}}{a_{11} - a_{12} - a_{21} + a_{22}}, \quad p = \frac{b_{22} - b_{21}}{b_{11} - b_{12} - b_{21} + b_{22}}$$

symbol		meaning	SI unit
`A`	in	row player's 2 x 2 payoffs	—
`B`	in	column player's 2 x 2 payoffs	—
`p_row1`	out	probability the row player plays row 1	—
`q_col1`	out	probability the column player plays column 1	—
`row_payoff`	out	row player's expected payoff	—
`col_payoff`	out	column player's expected payoff	—

Applies when: 2 x 2 matrices; no pure-strategy equilibrium (then the mixed one is unique).

```
$ optimization solve mixed_nash_2x2 "A=[[3, -1], [-2, 1]]" "B=[[-1, 2], [1, -1]]"
p_row1 = 0.4
q_col1 = 0.2857142857
row_payoff = 0.1428571429
col_payoff = 0.2
  by: Mixed-strategy Nash equilibrium of a 2 x 2 game  (indifference conditions in exact
arithmetic: p from the column player's payoffs, q from the row player's)
  assumes: 2 x 2 matrices; no pure-strategy equilibrium (then the mixed one is unique)
```

What the worked example means: This calculation determines the mixed-strategy Nash equilibrium for a two-by-two game where no pure strategy is stable. Setting probabilities to make each opponent indifferent between their choices shows that the row player should play their first strategy with probability 0.4, while the column player should play their first strategy with probability 0.285714. Consequently, the expected payoff for the row player is 0.142857 and for the column player is 0.2.

Pure-strategy Nash equilibria of a bimatrix game — `pure_nash_equilibria`

Every pure-strategy Nash equilibrium (a cell where each player's strategy is a best reply to the other's) of a two-player game given by the two players' payoff matrices.

$$a_{rs} = \max_k a_{ks}, \quad b_{rs} = \max_k b_{rk}$$

symbol		meaning	SI unit
`A`	in	row player's payoffs (rows = row strategies)	—
`B`	in	column player's payoffs (same shape)	—
`count`	out	number of pure equilibria	—
`equilibria`	out	all pure equilibria as [row, column], 1-based, in row-major order	—

Applies when: at most 8 strategies per player; weak best replies count (ties are equilibria).

```
$ optimization solve pure_nash_equilibria "A=[[3, 0], [5, 1]]" "B=[[3, 5], [0, 1]]"
count = 1
equilibria = [[2.0,2.0]]
  by: Pure-strategy Nash equilibria of a bimatrix game  (mutual best replies, cell by cell, in
exact arithmetic)
  assumes: at most 8 strategies per player; weak best replies count (ties are equilibria)
```

What the worked example means: This calculation determines all stable outcomes where neither player can improve their payoff by unilaterally changing their strategy. It scans every cell of the input matrices [3 0; 5 1] and [3 5; 0 1] to find mutual best replies, meaning the row player's choice maximizes their payoff given the column player's choice, and vice versa. The result reports the total number of such equilibria as 1 and lists them as coordinate pairs in [[2, 2]], using one-based indexing for clarity.

Two-person zero-sum game (value and optimal mixed strategies) — `zero_sum_game`

The value of a matrix game (row player maximises the payoff in the matrix, column player minimises it) and one optimal mixed strategy for each player.

$$v = \max_{x \in \Delta_m} \min_j \sum_i x_i a_{ij} = \min_{y \in \Delta_n} \max_i \sum_j a_{ij} y_j$$

symbol		meaning	SI unit
`A`	in	payoff matrix to the row player, rows = row strategies, columns = column strategies	—
`value`	out	value of the game	—
`row_strategy_example`	out	one optimal mixed strategy of the row player	—
`col_strategy_example`	out	one optimal mixed strategy of the column player	—

Applies when: at most 12 strategies per player.

```
$ optimization solve zero_sum_game "A=[[3, -1, 2], [-2, 4, 1], [0, 1, -1]]"
value = 1
row_strategy_example = [0.6,0.4,0.0]
col_strategy_example = [0.5,0.5,0.0]
  by: Two-person zero-sum game (value and optimal mixed strategies)  (the game as a pair of dual
linear programs on the shifted matrix, solved exactly (rational simplex); the pair of strategies is
the certificate (each guarantees the value))
  assumes: at most 12 strategies per player
```

What the worked example means: This calculation determines the equilibrium outcome for a two-person zero-sum game defined by the payoff matrix [3 -1 2; -2 4 1; 0 1 -1]. It computes the game's value, which represents the expected payoff to the row player when both participants use their optimal mixed strategies. The result includes one specific optimal probability distribution for the row player, shown as [0.6, 0.4, 0], and one for the column player, shown as [0.5, 0.5, 0]. These strategies guarantee that neither player can improve their expected outcome by unilaterally changing their approach.

Integer programming (8)

0/1 knapsack problem — `knapsack_01`

Choose items (each at most once) to maximise total value without exceeding the capacity: the maximum value, one optimal choice and its weight.

$$\max \sum_i v_i x_i \text{ s.t. } \sum_i w_i x_i \le W,\ x_i \in \{0,1\}$$

symbol		meaning	SI unit
`values`	in	value of each item	—
`weights`	in	weight of each item	—
`capacity`	in	knapsack capacity	—
`max_value`	out	maximum total value	—
`items_example`	out	one optimal set of items (1-based; ties to the lighter set)	—
`total_weight`	out	weight of that set	—

Applies when: values, weights, capacity >= 0; at most 40 items.

```
$ optimization solve knapsack_01 "values=[60, 100, 120]" "weights=[10, 20, 30]" capacity=50
max_value = 220
items_example = [2,3]
total_weight = 50
  by: 0/1 knapsack problem  (exact dynamic programming over (weight, value) states with Pareto
pruning, rational arithmetic)
  capacity = 50
  assumes: values, weights, capacity >= 0; at most 40 items
```

What the worked example means: This calculation determines the optimal subset of items to maximize total value without exceeding the given capacity. Using exact dynamic programming, it identifies that the best possible value is 220. This result corresponds to choosing the specific set of items listed as [2, 3], which together weigh 50. The solution respects the constraint that each item can be taken at most once and ensures no negative values or weights are involved.

Bin packing (minimum number of bins) — `bin_packing`

Pack items of given sizes into the fewest bins of a given capacity: the minimum number and one packing.

$$\min \#\text{bins} \ge \left\lceil \sum_j s_j / C \right\rceil$$

symbol		meaning	SI unit
`sizes`	in	size of each item	—
`capacity`	in	capacity of each bin	—
`min_bins`	out	minimum number of bins	—
`bin_of_item`	out	one optimal packing: bin number of each item	—
`lower_bound`	out	ceil(total size / capacity)	—

Applies when: at most 14 items; every item fits in a bin.

```
$ optimization solve bin_packing "sizes=[4, 8, 1, 4, 2, 1]" capacity=10
min_bins = 2
bin_of_item = [2,1,2,2,1,2]
lower_bound = 2
  by: Bin packing (minimum number of bins)  (exact depth-first search (largest item first,
symmetric bins skipped) with the volume bound)
  capacity = 10
  assumes: at most 14 items; every item fits in a bin
```

What the worked example means: This calculation determines the fewest containers needed to hold all items, resulting in a minimum of 2 bins. The process checks if each item fits into a bin and uses an exact search method to find an optimal packing arrangement, represented by [2, 1, 2, 2, 1, 2]. A theoretical floor for this count is established by dividing the total size of all items by the bin capacity, yielding a lower bound of 2. If the computed minimum equals this bound, the packing is perfectly efficient with no wasted space beyond what is unavoidable.

Chromatic number (minimum graph colouring) — `graph_coloring`

The fewest colours needed so that adjacent nodes get different colours (scheduling exams, assigning frequencies), and one such colouring.

$$\chi(G) = \min\{k : \exists\, c : V \to \{1..k\},\ c(u) \neq c(v)\ \forall uv \in E\}$$

symbol		meaning	SI unit
`edges`	in	edges 'A-B, B-C, ...' (undirected)	—
`chromatic_number`	out	minimum number of colours	—
`coloring_example`	out	one optimal colouring, in the order of 'nodes'	—
`nodes`	out	node order (first appearance)	—

Applies when: at most 14 nodes; no self-loops.

```
$ optimization solve graph_coloring "edges=A-B, B-C, C-A, C-D"
chromatic_number = 3
coloring_example = [2.0,3.0,1.0,2.0]
nodes = ["A","B","C","D"]
  by: Chromatic number (minimum graph colouring)  (exact backtracking for k = 1, 2, ... colours
(largest degree first, symmetric colours skipped))
  assumes: at most 14 nodes; no self-loops
```

What the worked example means: This calculation determines the chromatic number, which is the smallest integer 3 such that every node in the set [A, B, C, D] can be assigned one of these colors without any two connected nodes having the same color. The result includes a specific valid assignment [2, 3, 1, 2] derived from the input edges A-B, B-C, C-A, C-D. This value represents the optimal solution found by exhaustively checking possible colorings, providing both the count and an example configuration for resource allocation tasks like scheduling or frequency assignment.

Integer linear program (branch and bound) — `integer_program`

Optimise a linear objective over non-negative INTEGER points satisfying linear constraints: the optimal value and one optimal integer solution.

$$\max / \min \; c^{\top} x \quad \text{s.t. } Ax \; (\leq, \geq, =) \; b, \; x \in \mathbb{Z}^{n}_{\geq 0}$$

symbol		meaning	SI unit
`sense`	in	maximize or minimize	—
`c`	in	objective coefficients, one per variable	—
`A`	in	constraint coefficients, one row per constraint	—
`b`	in	right-hand sides, one per constraint	—
`relations`	in	relation of each constraint: <=, >= or = (default all <=) (optional)	—
`optimal_value`	out	optimal value over integer points	—
`x_one_optimal`	out	one optimal integer solution (the first found: depth-first, lowest-index fractional variable, round-down branch first)	—
`nodes`	out	branch-and-bound nodes explored	—

Applies when: all variables integer and >= 0; a bounded LP relaxation; at most 4000 branch-and-bound nodes.

```
$ optimization solve integer_program sense=maximize "c=[5, 4]" "A=[[6, 4], [1, 2]]" "b=[24, 6]"
"relations=<=,<="
optimal_value = 20
x_one_optimal = [4.0,0.0]
nodes = 5
  by: Integer linear program (branch and bound)  (LP-based branch and bound; every relaxation
solved exactly (rational simplex), so pruning and integrality tests are exact)
  sense = maximize
  relations = <=,<=
  assumes: all variables integer and >= 0; a bounded LP relaxation; at most 4000 branch-and-bound
nodes
```

What the worked example means: This calculation determines the best possible outcome for a linear objective function when all decision variables must be whole numbers. It takes the coefficients defining the goal and constraints, then systematically searches through potential solutions using branch and bound to ensure exactness. The result provides the maximum or minimum achievable value, represented here as 20, along with one specific set of variable values that achieves it.

Minimum vertex cover (and maximum independent set) — `minimum_vertex_cover`

The fewest nodes that touch every edge of a graph (placing guards or sensors), one such set, and the size of a largest set of pairwise non-adjacent nodes.

$$\tau(G) = \min\{|C| : C \cap \{u,v\} \neq \emptyset \; \forall uv \in E\}, \quad \alpha(G) = |V| - \tau(G)$$

symbol		meaning	SI unit
`edges`	in	edges 'A-B, B-C, ...' (undirected)	—
`min_cover_size`	out	size of a minimum vertex cover	—
`cover_example`	out	one minimum vertex cover	—
`max_independent_set_size`	out	size of a maximum independent set (nodes minus cover)	—

Applies when: at most 30 nodes; no self-loops; only nodes that appear in an edge count.

```
$ optimization solve minimum_vertex_cover "edges=A-B, A-C, A-D, B-C, D-E"
min_cover_size = 3
cover_example = ["A","C","E"]
max_independent_set_size = 2
  by: Minimum vertex cover (and maximum independent set)  (exact branch and bound (a node of
largest degree in the cover, or all its neighbours))
  assumes: at most 30 nodes; no self-loops; only nodes that appear in an edge count
```

What the worked example means: This calculation determines the smallest number of nodes needed to touch every edge in the graph, resulting in a minimum vertex cover of size 3. One such optimal set is identified as [A, C, E]. Additionally, the size of the largest group of nodes where no two are connected, known as the maximum independent set, is computed as 2. These values help understand efficient sensor placement or security coverage within the network structure.

Minimum-cost set cover — `set_cover`

Choose sets (each with a cost) that together contain every element, at least total cost.

$$\min \sum_i c_i x_i \text{ s.t. } \sum_{i \ni e} x_i \geq 1 \; \forall e, \; x \in \{0,1\}$$

symbol		meaning	SI unit
`sets`	in	incidence matrix: row i lists with 0/1 which elements set i contains	—
`costs`	in	cost of each set	—
`min_cost`	out	minimum total cost of a cover	—
`sets_example`	out	one optimal choice of sets (1-based)	—

Applies when: at most 18 sets; every element in some set.

```
$ optimization solve set_cover "sets=[[1, 1, 0, 0], [0, 1, 1, 0], [0, 0, 1, 1], [1, 0, 0, 1]]"
"costs=[3, 2, 3, 2]"
min_cost = 4
sets_example = [2,4]
  by: Minimum-cost set cover  (exhaustive enumeration of set combinations (exact))
  assumes: at most 18 sets; every element in some set
```

What the worked example means: This calculation determines the least expensive combination of sets that includes every required element. By exhaustively checking all possible combinations, it guarantees an exact solution where the total cost equals 4 when choosing sets [2, 4]. This result represents the optimal trade-off between coverage completeness and resource expenditure.

Unbounded knapsack problem — `knapsack_unbounded`

Choose any number of copies of each item type to maximise total value within the capacity.

$$V(c) = \max\left(V(c-1),\ \max_{i:w_i \le c} V(c-w_i) + v_i\right)$$

symbol		meaning	SI unit
`values`	in	value of each item type	—
`weights`	in	weight of each item type (whole numbers)	—
`capacity`	in	capacity (a whole number)	—
`max_value`	out	maximum total value	—
`copies_example`	out	number of copies of each type in one optimal choice	—

Applies when: positive whole-number weights; capacity a whole number <= 100000.

```
$ optimization solve knapsack_unbounded "values=[10, 40, 50, 70]" "weights=[1, 3, 4, 5]" capacity=8
max_value = 110
copies_example = [0.0,1.0,0.0,1.0]
  by: Unbounded knapsack problem  (dynamic programming over capacities)
  capacity = 8
  assumes: positive whole-number weights; capacity a whole number <= 100000
```

What the worked example means: Using dynamic programming over capacities, the model determines the best combination of item types to maximize total value without exceeding the limit. Given values [10, 40, 50, 70] and weights [1, 3, 4, 5], the optimal selection yields a total value of 110. One example configuration achieving this uses [0, 1, 0, 1] copies of each respective type.

Uncapacitated facility location — `facility_location`

Choose which facilities to open (each with a fixed cost) and serve every customer from its cheapest open facility, minimising fixed plus service cost.

$$\min_{S \neq \emptyset} \sum_{i \in S} f_i + \sum_{j} \min_{i \in S} c_{ij}$$

symbol		meaning	SI unit
`fixed`	in	fixed opening cost of each facility	—
`cost`	in	service cost matrix, rows = facilities, columns = customers	—
`min_cost`	out	minimum total cost	—
`open_example`	out	one optimal set of facilities to open (1-based)	—

Applies when: at most 15 facilities; unlimited capacity.

```
$ optimization solve facility_location "fixed=[10, 15, 12]" "cost=[[3, 8, 6, 9], [7, 2, 4, 3], [5,
6, 2, 8]]"
min_cost = 31
open_example = [2]
  by: Uncapacitated facility location  (exact enumeration of the facilities to open; each customer
assigned to its cheapest open facility)
  assumes: at most 15 facilities; unlimited capacity
```

What the worked example means: This calculation determines the optimal subset of facilities to open by enumerating all possible combinations, subject to a limit on the number of facilities. For each combination, every customer is assigned to the available facility with the lowest service cost. The method sums the fixed opening costs of the selected facilities and the resulting service costs for all customers. The output provides the global minimum total cost, which is 31, along with one example of an optimal facility set, denoted as [2]. This approach ensures that the trade-off between paying higher fixed costs to open more facilities versus incurring lower service costs is resolved optimally.

Inventory management (9)

Economic order quantity (EOQ) — `eoq`

The order size that minimises ordering plus holding cost for a steady demand, the minimum cost, the number of orders per period and the time between orders.

$$Q^* = \sqrt{\frac{2DS}{H}}, \quad TC^* = \sqrt{2DSH}$$

symbol		meaning	SI unit
`D`	in	demand per period (units, e.g. per year)	—
`S`	in	fixed cost per order	—
`H`	in	holding cost per unit per period	—
`Q`	out	economic order quantity	—
`total_cost`	out	minimum ordering + holding cost per period	—
`orders_per_period`	out	number of orders per period	—
`cycle_time`	out	time between orders (in periods)	—

Applies when: D, S, H > 0; constant demand, instant replenishment, no shortages.

```
$ optimization solve eoq D=1200 S=50 H=3
Q = 200
total_cost = 600
orders_per_period = 6
cycle_time = 0.1666666667
  by: Economic order quantity (EOQ)  (minimise D S / Q + H Q / 2 (at the optimum ordering cost
equals holding cost))
  D = 1200
  S = 50
  H = 3
  assumes: D, S, H > 0; constant demand, instant replenishment, no shortages
```

What the worked example means: This calculation determines the economic order quantity by balancing ordering and holding costs. Using a demand of 1200, an ordering cost of 50, and a holding cost of 3, the model finds that ordering 200 units minimizes expenses. At this optimum, the total cost is 600 per period, requiring 6 orders per period with a cycle time of 0.166667.

Economic production quantity (EPQ) — `epq_production_lot`

The production lot size that minimises setup plus holding cost when stock builds up gradually at a finite production rate.

$$Q^* = \sqrt{\frac{2DS}{H(1 - D/P)}}$$

symbol		meaning	SI unit
`D`	in	demand rate per period	—
`P`	in	production rate per period (> D)	—
`S`	in	setup cost per run	—
`H`	in	holding cost per unit per period	—
`Q`	out	economic production quantity	—
`max_inventory`	out	maximum inventory Q (1 - D/P)	—
`total_cost`	out	minimum setup + holding cost per period	—
`runs_per_period`	out	production runs per period	—

Applies when: P > D > 0; S, H > 0.

```
$ optimization solve epq_production_lot D=1000 S=100 H=4 P=2000
Q = 316.227766
max_inventory = 158.113883
total_cost = 632.455532
runs_per_period = 3.16227766
  by: Economic production quantity (EPQ)  (minimise D S/Q + H Q (1 - D/P)/2)
  D = 1000
  S = 100
  H = 4
  P = 2000
  assumes: P > D > 0; S, H > 0
```

What the worked example means: This calculation determines the economic production quantity, which is the ideal batch size that minimizes total inventory costs when items are manufactured gradually. By considering a demand rate of 1000, a production rate of 2000, a setup cost per run of 100, and a holding cost per unit of 4, the model finds that producing batches of 316.228 units results in a minimum total cost of 632.456 per period. This approach accounts for the fact that stock builds up over time rather than arriving instantly, allowing for simultaneous production and consumption.

EOQ with planned backorders — `eoq_planned_shortages`

The order quantity and the planned maximum backorder that minimise ordering, holding and shortage cost when unmet demand is backordered.

$$Q^* = \sqrt{\frac{2DS}{H} \cdot \frac{H+B}{B}}, \quad s^* = Q^* \frac{H}{H+B}$$

symbol		meaning	SI unit
`D`	in	demand per period	—
`S`	in	cost per order	—
`H`	in	holding cost per unit per period	—
`B`	in	backorder (shortage) cost per unit per period	—
`Q`	out	optimal order quantity	—
`max_backorder`	out	planned maximum backorder	—
`max_inventory`	out	maximum inventory level	—
`total_cost`	out	minimum total cost per period	—

Applies when: D, S, H, B > 0; backorders filled when the order arrives.

```
$ optimization solve eoq_planned_shortages D=1200 S=50 H=3 B=6
Q = 244.9489743
max_backorder = 81.64965809
max_inventory = 163.2993162
total_cost = 489.8979486
  by: EOQ with planned backorders  (minimise D S/Q + H (Q-s)^2/(2Q) + B s^2/(2Q) over Q and s)
  D = 1200
  S = 50
  H = 3
  B = 6
  assumes: D, S, H, B > 0; backorders filled when the order arrives
```

What the worked example means: This calculation determines the best inventory policy by minimizing total costs. The optimal order quantity is 244.949 and the planned maximum backorder is 81.6497. This results in a maximum inventory level of 163.299 and a minimum total cost per period of 489.898. These values balance ordering, holding, and shortage expenses under constant demand.

Newsvendor order quantity (discrete demand) — `newsvendor_discrete`

For demand taking listed values with given probabilities, the order quantity that maximises expected profit (the smallest demand value whose cumulative probability reaches the critical ratio) and that profit.

$$Q^* = \min\{d : F(d) \geq \tfrac{p-c}{p-s}\}$$

symbol		meaning	SI unit
`p`	in	selling price per unit	—
`c`	in	cost per unit	—
`demands`	in	possible demand values, increasing	—
`probs`	in	probability of each demand value (sum 1)	—
`s`	in	salvage value per unsold unit (default 0) (optional)	—

symbol		meaning	SI unit
`critical_ratio`	out	critical ratio (p - c)/(p - s)	—
`Q`	out	optimal order quantity (smallest optimal value)	—
`expected_profit`	out	maximum expected profit	—

Applies when: salvage < cost < price; probabilities sum to 1; order quantities restricted to the listed demand values.

```
$ optimization solve newsvendor_discrete p=10 c=4 "demands=[10, 20, 30, 40]" "probs=[0.1, 0.3, 0.4,
0.2]"
critical_ratio = 0.6
Q = 30
expected_profit = 130
  by: Newsvendor order quantity (discrete demand)  (cumulative distribution against the critical
ratio, in exact fractions)
  p = 10
  c = 4
  assumes: salvage < cost < price; probabilities sum to 1; order quantities restricted to the
listed demand values
```

What the worked example means: This calculation determines the best order size for discrete demand by comparing cumulative probabilities to the critical ratio. The result shows that ordering 30 units maximizes expected profit, which computes to 130. This quantity is chosen because it is the smallest demand level where the cumulative probability meets or exceeds the critical ratio of 0.6, balancing the cost of understocking against the cost of overstocking.

Newsvendor order quantity (normal demand) — `newsvendor_normal`

The single-period order quantity that maximises expected profit for normally distributed demand: the critical ratio, the optimal quantity and the expected profit.

$$F(Q^*) = \frac{p - c}{p - s}, \quad Q^* = \mu + z\sigma$$

symbol		meaning	SI unit
`p`	in	selling price per unit	—
`c`	in	cost per unit	—
`mu`	in	mean demand	—
`sigma`	in	standard deviation of demand	—
`s`	in	salvage value per unsold unit (default 0) (optional)	—
`critical_ratio`	out	critical ratio (p - c)/(p - s)	—
`Q`	out	optimal order quantity	—
`expected_profit`	out	expected profit at Q	—
`expected_shortage`	out	expected unmet demand at Q	—

Applies when: salvage < cost < price; sigma > 0; continuous normal demand.

```
$ optimization solve newsvendor_normal p=2 c=0.5 s=0.2 mu=100 sigma=20
critical_ratio = 0.8333333333
Q = 119.3484313
expected_profit = 141.0053661
expected_shortage = 1.772280259
  by: Newsvendor order quantity (normal demand)  (critical fractile: F(Q) = (p - c)/(p - s);
expected profit from the standard normal loss function)
  p = 2
  c = 0.5
  s = 0.2
  mu = 100
  sigma = 20
  assumes: salvage < cost < price; sigma > 0; continuous normal demand
```

What the worked example means: This calculation determines the best single-period order size for normally distributed demand. Comparing the cost of understocking against overstocking gives a critical ratio of 0.833333. This ratio dictates that you should order 119.348 units to maximize returns. At this level, the expected profit reaches 141.005, while any unmet demand averages 1.77228 units.

Newsvendor order quantity (uniform demand) — `newsvendor_uniform`

The single-period order quantity that maximises expected profit when demand is uniform on [a, b].

$$Q^* = a + \frac{p - c}{p - s}(b - a)$$

symbol		meaning	SI unit
`p`	in	selling price per unit	—
`c`	in	cost per unit	—
`lo`	in	lowest possible demand a	—
`hi`	in	highest possible demand b	—
`s`	in	salvage value per unsold unit (default 0) (optional)	—
`critical_ratio`	out	critical ratio (p - c)/(p - s)	—
`Q`	out	optimal order quantity a + CR (b - a)	—
`expected_profit`	out	expected profit at Q	—

Applies when: salvage < cost < price; 0 <= a < b.

```
$ optimization solve newsvendor_uniform p=10 c=4 lo=100 hi=300
critical_ratio = 0.6
Q = 220
expected_profit = 960
  by: Newsvendor order quantity (uniform demand)  (critical fractile of the uniform distribution)
  p = 10
  c = 4
  lo = 100
  hi = 300
  assumes: salvage < cost < price; 0 <= a < b
```

What the worked example means: The optimal order quantity is 220, derived from a critical ratio of 0.6.

Order quantity with quantity discounts (all-units) — `eoq_quantity_discount`

With all-units price breaks, the order quantity that minimises purchase + ordering + holding cost, the price paid and the minimum total cost per period.

$$TC(Q) = Dp_k + \frac{DS}{Q} + \frac{ip_kQ}{2}$$

symbol		meaning	SI unit
`D`	in	demand per period	—
`S`	in	cost per order	—
`i_rate`	in	holding cost as a fraction of the unit price per period (0.2 = 20%)	—
`breaks`	in	minimum order quantity of each price tier, increasing, the first 0	—
`prices`	in	unit price of each tier	—
`order_quantity`	out	optimal order quantity	—
`total_cost`	out	minimum total cost per period (purchase + ordering + holding)	—
`unit_price`	out	unit price paid	—

Applies when: D, S, i > 0; breaks increasing from 0; ties broken toward the lower-quantity tier.

```
$ optimization solve eoq_quantity_discount D=5000 S=49 i_rate=0.2 "breaks=[0, 1000, 2000]" "prices=
[5, 4.8, 4.75]"
order_quantity = 1000
total_cost = 24725
unit_price = 4.8
  by: Order quantity with quantity discounts (all-units)  (EOQ per tier at its price, raised to the
tier minimum when below it; tiers whose EOQ exceeds their upper limit are dominated; least total
cost)
  D = 5000
  S = 49
  i_rate = 0.2
  assumes: D, S, i > 0; breaks increasing from 0; ties broken toward the lower-quantity tier
```

What the worked example means: This calculation determines the best order size when suppliers offer price breaks for larger quantities. Evaluating the total cost, which includes purchase, ordering, and holding expenses, at each tier's minimum requirement or its economic order quantity, shows that ordering 1000 units minimizes the overall expense to 24725. The unit price paid for this volume is 4.8.

Periodic-review order-up-to level — `order_up_to_level`

The target inventory position S of a periodic-review system that covers demand over the review period plus the lead time with a given cycle service level.

$$S = \bar{d}(R + L) + z_{SL}\,\sigma_d\sqrt{R + L}$$

symbol		meaning	SI unit
`d`	in	average demand per period	—
`sigma`	in	standard deviation of demand per period	—
`R`	in	review period (periods)	—
`L`	in	lead time (periods)	—
`SL`	in	cycle service level, e.g. 0.95	—
`order_up_to_level`	out	order-up-to level S	—
`safety_stock`	out	safety stock z sigma sqrt(R + L)	—
`z`	out	standard normal quantile	—

Applies when: 0.5 <= SL < 1; independent normal demand per period.

```
$ optimization solve order_up_to_level d=20 sigma=4 R=7 L=2 SL=0.95
order_up_to_level = 199.7382435
safety_stock = 19.73824352
z = 1.644853627
  by: Periodic-review order-up-to level  (S = d (R + L) + z sigma sqrt(R + L))
  d = 20
  sigma = 4
  R = 7
  L = 2
  SL = 0.95
  assumes: 0.5 <= SL < 1; independent normal demand per period
```

What the worked example means: This calculation determines the target inventory position for a periodic-review system. It combines the average expected demand over the combined review period and lead time, calculated as 20 times the sum of 7 and 2, with a buffer known as safety stock. The safety stock, valued at 19.7382, is derived from the standard deviation of demand 4 scaled by the normal quantile 1.64485 corresponding to the service level 0.95. The resulting order-up-to level of 199.738 ensures that customer demand is met with the specified probability while accounting for variability in both demand and timing.

Reorder point with safety stock — `reorder_point`

The stock level at which to reorder so that demand during the lead time is covered with a given cycle service level, for normally distributed demand per period.

$$ROP = \bar{d}L + z_{SL}\,\sigma_d\sqrt{L}$$

symbol		meaning	SI unit
`d`	in	average demand per period	—
`L`	in	lead time in periods	—
`sigma`	in	standard deviation of demand per period	—
`SL`	in	cycle service level (probability of no stockout per cycle, e.g. 0.95)	—
`reorder_point`	out	reorder point	—
`safety_stock`	out	safety stock z sigma sqrt(L)	—
`z`	out	standard normal quantile of the service level	—

symbol		meaning	SI unit
`lead_time_demand`	out	expected demand during the lead time	—

Applies when: 0.5 <= SL < 1; independent normal demand per period.

```
$ optimization solve reorder_point d=40 sigma=8 L=9 SL=0.95
reorder_point = 399.476487
safety_stock = 39.47648705
z = 1.644853627
lead_time_demand = 360
  by: Reorder point with safety stock  (z = Phi^{-1}(SL); ROP = d L + z sigma sqrt(L))
  d = 40
  sigma = 8
  L = 9
  SL = 0.95
  assumes: 0.5 <= SL < 1; independent normal demand per period
```

What the worked example means: This calculation determines the inventory level that triggers a new order to achieve a cycle service level of 0.95. The result, 399.476, consists of two parts: the expected demand during the lead time, which is 360, and the safety stock buffer, calculated as 39.4765. This buffer accounts for demand variability using a standard normal quantile of 1.64485, ensuring that stockouts are unlikely under normal conditions.

Linear programming (3)

Linear program (simplex method) — `linear_program`

Optimise a linear objective subject to linear constraints with non-negative variables: the optimal value, an optimal solution (stated as unique only when it is) and the shadow prices.

$$\max / \min \; c^{\top} x \quad \text{s.t. } Ax \; (\leq, \geq, =) \; b, \; x \geq 0$$

symbol		meaning	SI unit
`sense`	in	maximize or minimize	—
`c`	in	objective coefficients, one per variable	—
`A`	in	constraint coefficients, one row per constraint	—
`b`	in	right-hand sides, one per constraint	—
`relations`	in	relation of each constraint: <=, >= or = (default all <=) (optional)	—
`optimal_value`	out	optimal value of the objective	—
`x`	out	the optimal solution, variables in order of first appearance (only when unique; otherwise x_one_optimal)	—
`solution_unique`	out	yes when the optimal solution is unique	—
`shadow_prices`	out	change of the optimal value per unit increase of each right-hand side (only when unique; otherwise dual_one_optimal)	—

Applies when: all variables >= 0; the LP is feasible and bounded (otherwise INVALID_INPUT says which); at most 12 variables and constraints.

```
$ optimization solve linear_program sense=maximize "c=[3, 5]" "A=[[1, 0], [0, 2], [3, 2]]" "b=[4,
12, 18]" "relations=<=,<=,<="
optimal_value = 36
x = [2.0,6.0]
solution_unique = yes
shadow_prices = [0.0,1.5,1.0]
  by: Linear program (simplex method)  (two-phase simplex in exact rational arithmetic with Bland's
anti-cycling rule; optimality certified by a dual solution (primal feasibility, dual feasibility,
equal objective values); uniqueness proved by an auxiliary LP over the optimal face)
  sense = maximize
  relations = <=,<=,<=
  assumes: all variables >= 0; the LP is feasible and bounded (otherwise INVALID_INPUT says which);
at most 12 variables and constraints
```

In the worked example, sense = maximize, c = [3, 5], A = [1 0; 0 2; 3 2], b = [4, 12, 18], relations = <=,<=,<= go through the equation above and give optimal_value = 36, x = [2, 6], solution_unique = yes, shadow_prices = [0, 1.5, 1]. It applies only when all variables >= 0; the LP is feasible and bounded (otherwise INVALID_INPUT says which); at most 12 variables and constraints: outside that it says what is wrong instead of returning a number.

Shadow prices of a linear program — `lp_shadow_prices`

The shadow price (dual value) of each constraint: the change of the optimal value per unit increase of its right-hand side, when it is uniquely defined.

$$y^\top = c_B^\top B^{-1}, \qquad \frac{\partial z^*}{\partial b_i} = y_i$$

symbol		meaning	SI unit
`sense`	in	maximize or minimize	—
`c`	in	objective coefficients, one per variable	—
`A`	in	constraint coefficients, one row per constraint	—
`b`	in	right-hand sides, one per constraint	—
`relations`	in	relation of each constraint: <=, >= or = (default all <=) (optional)	—
`shadow_prices`	out	shadow price of each constraint, in order	—
`optimal_value`	out	optimal value of the objective	—
`x`	out	the optimal solution (only when unique; otherwise x_one_optimal)	—
`solution_unique`	out	yes when the optimal solution is unique	—

Applies when: all variables >= 0; a finite optimum; unique dual solution (otherwise INVALID_INPUT).

```
$ optimization solve lp_shadow_prices sense=maximize "c=[3, 5]" "A=[[1, 0], [0, 2], [3, 2]]" "b=[4,
12, 18]" "relations=<=,<=,<="
optimal_value = 36
x = [2.0,6.0]
solution_unique = yes
shadow_prices = [0.0,1.5,1.0]
  by: Shadow prices of a linear program  (exact simplex; the dual solution y = c_B B^{-1};
uniqueness of the dual proved by an auxiliary LP over the dual optimal face)
  sense = maximize
  relations = <=,<=,<=
  assumes: all variables >= 0; a finite optimum; unique dual solution (otherwise INVALID_INPUT)
```

What the worked example means: This calculation determines the shadow prices for each constraint, represented as [0, 1.5, 1]. These values indicate the rate of change in the optimal objective value, 36, with respect to a one-unit increase in the corresponding right-hand side parameters. The result is derived from the dual solution associated with the optimal basis, providing insight into the marginal worth of resources defined by the constraints.

Two-variable linear program (graphical corner-point method) — `lp_two_variable_graphical`

Solve a linear program in two variables by evaluating the objective at every corner point of the feasible region.

$$z^* = \max_{v \in \text{corners}} c^\top v$$

symbol		meaning	SI unit
`sense`	in	maximize or minimize	—
`c`	in	objective coefficients, one per variable	—
`A`	in	constraint coefficients, one row per constraint	—
`b`	in	right-hand sides, one per constraint	—
`relations`	in	relation of each constraint: <=, >= or = (default all <=) (optional)	—
`optimal_value`	out	optimal value of the objective	—
`x`	out	first variable at the optimum (only when the optimum is unique)	—
`y`	out	second variable at the optimum (only when the optimum is unique)	—
`optimal_corners`	out	every corner point at which the optimum is attained	—
`corner_points`	out	every corner point of the feasible region	—
`solution_unique`	out	yes when exactly one point is optimal	—

Applies when: exactly two variables, both >= 0; a feasible region and a finite optimum.

```
$ optimization solve lp_two_variable_graphical sense=maximize "c=[3, 5]" "A=[[1, 0], [0, 2], [3,
2]]" "b=[4, 12, 18]" "relations=<=,<=,<="
optimal_value = 36
optimal_corners = [[2.0,6.0]]
corner_points = [[0.0,0.0],[0.0,6.0],[2.0,6.0],[4.0,0.0],[4.0,3.0]]
x = 2
y = 6
solution_unique = yes
  by: Two-variable linear program (graphical corner-point method)  (intersect every pair of
boundary lines (exact fractions), keep the feasible corners, evaluate the objective; unboundedness
detected from the extreme rays of the recession cone)
  sense = maximize
  relations = <=,<=,<=
  assumes: exactly two variables, both >= 0; a feasible region and a finite optimum
```

What the worked example means: This calculation determines the best outcome for a two-variable linear program by checking every corner of the feasible region. It uses objective coefficients [3, 5] and constraint data [1 0; 0 2; 3 2], [4, 12, 18], and <=,<=,<= to find valid corners [[0, 0], [0, 6], [2, 6], [4, 0], [4, 3]]. The final result shows that the maximum or minimum value 36 occurs at specific points, indicating whether the solution is unique based on yes.

Multi-criteria decision analysis (6)

AHP consistency ratio — `ahp_consistency_ratio`

How consistent a set of pairwise comparisons is: CI = (lambda_max - n)/(n - 1) and CR = CI / RI (Saaty's random index); CR below 0.10 is conventionally acceptable.

$$CI = \frac{\lambda_{\max} - n}{n - 1}, \quad CR = \frac{CI}{RI_n}$$

symbol		meaning	SI unit
`judgments`	in	upper-triangle pairwise judgments a12, a13, ..., a23, ... (Saaty scale; a_ji = 1/a_ij)	—
`CR`	out	consistency ratio	—
`CI`	out	consistency index	—
`lambda_max`	out	principal eigenvalue	—
`weights`	out	priority weights	—
`acceptable`	out	yes when CR < 0.10	—

Applies when: 3 to 10 criteria; RI = 0.58, 0.90, 1.12, 1.24, 1.32, 1.41, 1.45, 1.49 for n = 3..10 (Saaty).

```
$ optimization solve ahp_consistency_ratio "judgments=[2, 4, 2]"
CR = 0
CI = 0
lambda_max = 3
weights = [0.5714285714285714,0.2857142857142857,0.14285714285714285]
acceptable = yes
  by: AHP consistency ratio  (principal eigenpair by power iteration (certified as an eigenpair),
then CI and CR)
  assumes: 3 to 10 criteria; RI = 0.58, 0.90, 1.12, 1.24, 1.32, 1.41, 1.45, 1.49 for n = 3..10
(Saaty)
```

What the worked example means: This calculation determines how logically consistent your pairwise comparisons are. It derives the principal eigenvalue, denoted as 3, from the judgment matrix to compute the consistency index, or 0. This index is then divided by a random index specific to the number of criteria to yield the final consistency ratio, 0. A value below zero point one suggests that the derived priority weights, [0.571429, 0.285714, 0.142857], are reliable for decision-making.

AHP priority vector (principal eigenvector) — `ahp_priority_vector`

The priority weights of criteria compared in pairs (analytic hierarchy process): the normalised principal eigenvector of the reciprocal comparison matrix, and its eigenvalue lambda_max.

$$Pw = \lambda_{\max} w, \quad \sum_i w_i = 1$$

symbol		meaning	SI unit
`judgments`	in	upper-triangle pairwise judgments a12, a13, ..., a23, ... (Saaty scale; a_ji = 1/a_ij)	—
`weights`	out	priority weights (sum 1)	—
`lambda_max`	out	principal eigenvalue	—
`best`	out	highest-priority criterion, 1-based	—

Applies when: judgments > 0; 2 to 10 criteria.

```
$ optimization solve ahp_priority_vector "judgments=[2, 4, 2]"
weights = [0.5714285714285714,0.2857142857142857,0.14285714285714285]
lambda_max = 3
best = 1
  by: AHP priority vector (principal eigenvector)  (power iteration on the positive reciprocal
matrix; certified as an eigenpair (P w = lambda w, w > 0: the Perron vector))
  assumes: judgments > 0; 2 to 10 criteria
```

In the worked example, judgments = [2, 4, 2] go through the equation above and give weights = [0.5714285714, 0.2857142857, 0.1428571429], lambda_max = 3, best = 1. It applies only when judgments > 0; 2 to 10 criteria: outside that it says what is wrong instead of returning a number.

AHP weights by the geometric mean method — `ahp_geometric_mean`

Priority weights from pairwise comparisons as normalised row geometric means (the logarithmic least-squares solution).

$$w_i = \frac{(\prod_j a_{ij})^{1/n}}{\sum_k (\prod_j a_{kj})^{1/n}}$$

symbol		meaning	SI unit
`judgments`	in	upper-triangle pairwise judgments a12, a13, ..., a23, ... (Saaty scale; a_ji = 1/a_ij)	—
`weights`	out	priority weights (sum 1)	—
`max_weight`	out	largest weight	—
`best`	out	highest-priority criterion, 1-based	—

Applies when: judgments > 0; 2 to 10 criteria.

```
$ optimization solve ahp_geometric_mean "judgments=[2, 4, 2]"
weights = [0.5714285714285714,0.2857142857142857,0.14285714285714285]
max_weight = 0.5714285714
best = 1
  by: AHP weights by the geometric mean method  (row geometric means, normalised)
  assumes: judgments > 0; 2 to 10 criteria
```

What the worked example means: This calculation derives priority weights from pairwise judgments using row geometric means. Given the input judgments, the method computes the geometric mean of each row in the comparison matrix and normalizes these values so they sum to one. The resulting weights are [0.571429, 0.285714, 0.142857]. Among these, the largest weight is 0.571429, which corresponds to the highest-priority criterion identified as 1. This approach provides a logarithmic least-squares solution that reflects the relative importance of each criterion based on the provided comparisons.

TOPSIS ranking (closeness to the ideal solution) — `topsis`

Rank alternatives scored on several criteria by their relative closeness to the ideal and distance from the anti-ideal solution (vector normalisation, weighted Euclidean distances).

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-}, \quad D_i^{\pm} = \left\| w \odot r_i - v^{\pm} \right\|_2$$

symbol		meaning	SI unit
`X`	in	decision matrix, rows = alternatives, columns = criteria (all values > 0)	—
`w`	in	criterion weights (> 0; normalised to sum 1)	—
`types`	in	criterion types: 1 = benefit (larger is better), -1 = cost (smaller is better)	—
`scores`	out	score of each alternative	—
`best_score`	out	the highest score	—
`best`	out	best alternative, 1-based (on a tie the first listed)	—

Applies when: values > 0; weights > 0 (normalised to sum 1); types 1 or -1.

```
$ optimization solve topsis "X=[[1, 1], [2, 2]]" "w=[1, 1]" "types=[1, 1]"
scores = [0.0,1.0]
best_score = 1
best = 2
  by: TOPSIS ranking (closeness to the ideal solution)  (vector normalisation r = x / ||x_col||,
weighted; ideal = best per criterion, anti-ideal = worst; C = D- / (D+ + D-))
  assumes: values > 0; weights > 0 (normalised to sum 1); types 1 or -1
```

What the worked example means: This calculation ranks alternatives by their relative closeness to an ideal solution. Using vector normalization and weighted Euclidean distances, it computes a score for each option based on how close it is to the best possible performance across all criteria and how far it is from the worst. The highest score, 1, belongs to alternative 2, indicating it is the most balanced choice given the weights and criteria types.

Weighted product model — `weighted_product_model`

Score alternatives by the product of linearly normalised criterion values raised to the criterion weights.

$$P_i = \prod_j n_{ij}^{w_j}$$

symbol		meaning	SI unit
`X`	in	decision matrix, rows = alternatives, columns = criteria (all values > 0)	—
`w`	in	criterion weights (> 0; normalised to sum 1)	—
`types`	in	criterion types: 1 = benefit (larger is better), -1 = cost (smaller is better)	—
`scores`	out	score of each alternative	—
`best_score`	out	the highest score	—
`best`	out	best alternative, 1-based (on a tie the first listed)	—

Applies when: values > 0; weights > 0 (normalised to sum 1); types 1 or -1.

```
$ optimization solve weighted_product_model "X=[[4, 2], [2, 4]]" "w=[0.5, 0.5]" "types=[1, 1]"
scores = [0.7071067811865476,0.7071067811865476]
best_score = 0.7071067812
best = 1
  by: Weighted product model  (linear normalisation (x / max for benefits, min / x for costs), then
P_i = prod_j n_ij^{w_j})
  assumes: values > 0; weights > 0 (normalised to sum 1); types 1 or -1
```

What the worked example means: This algorithm evaluates alternatives by computing the product of linearly normalized criterion values raised to their respective weights. It takes the decision matrix [4 2; 2 4], criterion weights [0.5, 0.5], and types [1, 1] as inputs. The resulting scores [0.707107, 0.707107] indicate relative performance, where higher values are better. The top-ranked alternative is identified as 1 with a maximum score of 0.707107. This multiplicative approach emphasizes that strong performance across all criteria is required for a high overall ranking.

Weighted sum model (simple additive weighting) — `weighted_sum_model`

Score alternatives by the weighted sum of linearly normalised criterion values (benefit x / max, cost min / x) and pick the best.

$$S_i = \sum_j w_j n_{ij}, \quad n_{ij} = \frac{x_{ij}}{\max_i x_{ij}} \text{ or } \frac{\min_i x_{ij}}{x_{ij}}$$

symbol		meaning	SI unit
`X`	in	decision matrix, rows = alternatives, columns = criteria (all values > 0)	—
`w`	in	criterion weights (> 0; normalised to sum 1)	—
`types`	in	criterion types: 1 = benefit (larger is better), -1 = cost (smaller is better)	—
`scores`	out	score of each alternative	—
`best_score`	out	the highest score	—
`best`	out	best alternative, 1-based (on a tie the first listed)	—

Applies when: values > 0; weights > 0 (normalised to sum 1); types 1 or -1.

```
$ optimization solve weighted_sum_model "X=[[250, 16, 12], [200, 16, 8], [300, 32, 16]]" "w=[0.4,
0.3, 0.3]" "types=[-1, 1, 1]"
scores = [0.6950000000000001,0.7,0.8666666666666667]
best_score = 0.8666666667
best = 3
  by: Weighted sum model (simple additive weighting)  (linear normalisation (x / max for benefits,
min / x for costs), then S_i = sum_j w_j n_ij)
  assumes: values > 0; weights > 0 (normalised to sum 1); types 1 or -1
```

What the worked example means: This method calculates scores for each option by normalizing criterion values based on whether they are benefits or costs, then summing these weighted values. The resulting highest score belongs to the top-ranked choice, identified as 3, which achieved a total value of 0.866667. This process relies on positive input data and weights that sum to one to ensure a valid comparison across different scales.

Network optimisation (12)

All-pairs shortest distances (Floyd-Warshall) — `all_pairs_shortest_paths`

The table of shortest distances between every pair of nodes, and the graph's diameter.

$$d_{ij} \leftarrow \min(d_{ij},\, d_{ik} + d_{kj})$$

symbol		meaning	SI unit
`edges`	in	weighted edges 'A-B 4, ...'	—
`directed`	in	true for a directed graph, false for undirected	—
`distances`	out	shortest-distance matrix (rows and columns in the order of 'nodes')	—
`nodes`	out	node order (first appearance)	—
`diameter`	out	largest shortest distance	—

Applies when: every node reaches every other; no negative cycle; at most 40 nodes.

```
$ optimization solve all_pairs_shortest_paths "edges=A-B 3, B-C 4, A-C 9, C-D 2" directed=false
nodes = ["A","B","C","D"]
distances = [[0.0,3.0,7.0,9.0],[3.0,0.0,4.0,6.0],[7.0,4.0,0.0,2.0],[9.0,6.0,2.0,0.0]]
diameter = 9
  by: All-pairs shortest distances (Floyd-Warshall)  (Floyd-Warshall in exact arithmetic)
  assumes: every node reaches every other; no negative cycle; at most 40 nodes
```

What the worked example means: This calculation determines the shortest distance between every pair of nodes using the Floyd-Warshall algorithm. The resulting matrix, [[0, 3, 7, 9], [3, 0, 4, 6], [7, 4, 0, 2], [9, 6, 2, 0]], provides a complete map of connectivity costs based on the input edges, A-B (3), B-C (4), A-C (9), C-D (2). The maximum value in this matrix is identified as the graph's diameter, 9, which indicates the greatest minimum effort required to travel between any two points in the network.

Assignment problem (Hungarian method), maximum total value — `assignment_max_profit`

Assign each row (worker) to a different column (task) so that the total value is as large as possible: the optimum and an optimal assignment (stated as THE assignment only when unique).

$$\min_{\sigma} \sum_{i} c_{i\sigma(i)} = \max \left\{ \sum_{i} u_i + \sum_{j} v_j : u_i + v_j \le c_{ij} \right\}$$

symbol		meaning	SI unit
`C`	in	value matrix, rows = workers, columns = tasks (square)	—
`max_value`	out	optimal total value	—
`assignment`	out	column assigned to each row, 1-based (only when unique; otherwise assignment_example)	—
`assignment_unique`	out	yes when the optimal assignment is unique	—
`dual_u`	out	row potentials	—
`dual_v`	out	column potentials (u_i + v_j bounds every cell: the optimality certificate)	—

Applies when: square matrix, at most 12 x 12 (pad with zeros for unbalanced problems).

```
$ optimization solve assignment_max_profit "C=[[9, 2, 7], [6, 4, 3], [5, 8, 1]]"
max_value = 21
assignment = [3,1,2]
assignment_unique = yes
dual_u = [7.0,4.0,8.0]
dual_v = [2.0,0.0,0.0]
  by: Assignment problem (Hungarian method), maximum total value  (Hungarian method (shortest
augmenting paths with potentials) in exact arithmetic; uniqueness by forbidding each chosen cell)
  assumes: square matrix, at most 12 x 12 (pad with zeros for unbalanced problems)
```

What the worked example means: The Hungarian method computes the maximum total value for assigning workers to tasks using the input matrix [9 2 7; 6 4 3; 5 8 1]. The resulting optimal total value is 21. If the solution is unique, indicated by yes, the specific assignment is given as [3, 1, 2]; otherwise, an example of a

valid optimal assignment is provided. This process ensures every worker is assigned exactly one distinct task while maximizing the sum of their values.

Assignment problem (Hungarian method), minimum total cost — `assignment_min_cost`

Assign each row (worker) to a different column (task) so that the total cost is as small as possible: the optimum and an optimal assignment (stated as THE assignment only when unique).

$$\min_{\sigma} \sum_{i} c_{i\sigma(i)} = \max \left\{ \sum_{i} u_i + \sum_{j} v_j : u_i + v_j \le c_{ij} \right\}$$

symbol		meaning	SI unit
`C`	in	cost matrix, rows = workers, columns = tasks (square)	—
`min_cost`	out	optimal total cost	—
`assignment`	out	column assigned to each row, 1-based (only when unique; otherwise assignment_example)	—
`assignment_unique`	out	yes when the optimal assignment is unique	—
`dual_u`	out	row potentials	—
`dual_v`	out	column potentials (u_i + v_j bounds every cell: the optimality certificate)	—

Applies when: square matrix, at most 12 x 12 (pad with zeros for unbalanced problems).

```
$ optimization solve assignment_min_cost "C=[[9, 2, 7], [6, 4, 3], [5, 8, 1]]"
min_cost = 9
assignment = [2,1,3]
assignment_unique = yes
dual_u = [4.0,6.0,4.0]
dual_v = [0.0,-2.0,-3.0]
  by: Assignment problem (Hungarian method), minimum total cost  (Hungarian method (shortest
augmenting paths with potentials) in exact arithmetic; uniqueness by forbidding each chosen cell)
  assumes: square matrix, at most 12 x 12 (pad with zeros for unbalanced problems)
```

In the worked example, C = [9 2 7; 6 4 3; 5 8 1] go through the equation above and give min_cost = 9, assignment = [2, 1, 3], assignment_unique = yes, dual_u = [4, 6, 4]. It applies only when square matrix, at most 12 x 12 (pad with zeros for unbalanced problems): outside that it says what is wrong instead of returning a number.

Maximum bipartite matching — `bipartite_matching`

The largest number of pairs that can be formed between two groups (workers and jobs, students and projects) using only allowed pairs, with one maximum matching and a vertex cover proving it maximum.

$$\nu(G) = \tau(G) \text{ for bipartite } G$$

symbol		meaning	SI unit
`edges`	in	allowed pairs 'left-right', e.g. 'W1-J1, W1-J2, W2-J1'	—
`max_matching`	out	size of a maximum matching	—

symbol		meaning	SI unit
`matching_example`	out	one maximum matching	—
`vertex_cover`	out	a minimum vertex cover of the same size (Konig's theorem: the certificate)	—

Applies when: bipartite: the first node of each pair is on the left, the second on the right.

```
$ optimization solve bipartite_matching "edges=W1-J1, W1-J2, W2-J1, W3-J2, W3-J3"
max_matching = 3
matching_example = ["W1-J2","W2-J1","W3-J3"]
vertex_cover = ["W1","W2","W3"]
  by: Maximum bipartite matching  (augmenting paths (Kuhn); minimum vertex cover from alternating
reachability (Konig))
  assumes: bipartite: the first node of each pair is on the left, the second on the right
```

In the worked example, edges = W1-J1, W1-J2, W2-J1, W3-J2, W3-J3 go through the equation above and give max_matching = 3, matching_example = [W1-J2, W2-J1, W3-J3], vertex_cover = [W1, W2, W3]. It applies only when bipartite: the first node of each pair is on the left, the second on the right: outside that it says what is wrong instead of returning a number.

Maximum flow and minimum cut (Edmonds-Karp) — `max_flow`

The largest flow that can be sent from a source to a sink through a network of capacitated arcs, a flow achieving it and the minimum cut that proves it optimal.

$$\max |f| = \min_{(S,\bar{S})} c(S, \bar{S})$$

symbol		meaning	SI unit
`edges`	in	directed arcs with capacities 'S-A 10, A-T 5, ...'	—
`source`	in	source node	—
`target`	in	sink node	—
`max_flow`	out	maximum flow value	—
`min_cut_source_side`	out	nodes on the source side of a minimum cut (those reachable in the final residual network)	—
`edge_flows`	out	flow on each arc, in input order (one maximum flow)	—

Applies when: directed arcs; capacities >= 0.

Run `optimization info max_flow` for its inputs, then `optimization solve max_flow name=value ...`.

What the worked example means: This calculation determines the largest amount of flow that can be sent from the source node S to the target node T through a network of directed arcs with specific capacities. The result, 15, represents the bottleneck capacity of the system, which is mathematically equal to the sum of capacities across the minimum cut defined by the set of nodes [S]. Additionally, the specific distribution of flow along each arc in the network is provided as [10, 5, 5, 5, 10], showing how the total volume is routed while respecting all capacity constraints and conservation laws.

Maximum-weight matching (general graph) — `max_weight_matching`

Pair up nodes of a graph (each node at most once) so that the total weight of the chosen edges is as large as possible: the maximum weight and one optimal matching.

$$\max_{M \text{ matching}} \sum_{e \in M} w_e$$

symbol		meaning	SI unit
`edges`	in	undirected weighted edges 'A-B 4, ...'	—
`max_weight`	out	maximum total weight of a matching	—
`matching_example`	out	one maximum-weight matching	—

Applies when: at most 16 nodes; edges of weight <= 0 are never worth choosing.

```
$ optimization solve max_weight_matching "edges=A-B 6, B-C 5, C-D 6, A-D 2, A-C 4"
max_weight = 12
matching_example = ["A-B","C-D"]
  by: Maximum-weight matching (general graph)  (exact dynamic programming over node subsets)
  assumes: at most 16 nodes; edges of weight <= 0 are never worth choosing
```

What the worked example means: This calculation determines the optimal way to pair nodes in an undirected graph such that each node appears in at most one pair and the sum of the selected edge weights is maximized. The result, 12, represents this highest possible total value, while [A-B, C-D] lists the specific edges that form this optimal configuration. This approach uses exact dynamic programming over subsets of nodes, which is feasible here because the graph contains no more than sixteen nodes.

Minimum spanning tree (Kruskal's algorithm) — `minimum_spanning_tree`

The least total weight of edges connecting all nodes of an undirected graph, and the tree (stated as THE tree only when the minimum spanning tree is unique).

$$\min_{T \text{ spanning tree}} \sum_{e \in T} w_e$$

symbol		meaning	SI unit
`edges`	in	undirected weighted edges 'A-B 4, ...'	—
`total_weight`	out	weight of a minimum spanning tree	—
`tree_edges`	out	the minimum spanning tree (only when unique; otherwise tree_edges_example)	—
`tree_unique`	out	yes when the minimum spanning tree is unique	—

Applies when: connected undirected graph.

```
$ optimization solve minimum_spanning_tree "edges=A-B 4, A-C 1, B-C 2, B-D 5, C-D 8"
total_weight = 8
tree_unique = yes
tree_edges = ["A-C","B-C","B-D"]
  by: Minimum spanning tree (Kruskal's algorithm)  (Kruskal (edges by weight, union-find);
uniqueness by the cycle property; total re-checked by Prim)
  assumes: connected undirected graph
```

What the worked example means: This calculation determines the subset of edges that connects all nodes in an undirected graph with the lowest possible total cost. Sorting edges by weight and adding them if they do not form a cycle gives the optimal structure. The resulting total weight is 8, and the specific connections forming this tree are listed as [A-C, B-C, B-D]. If the minimum spanning tree is unique, it represents the only way to achieve this minimal connectivity cost.

Minimum-cost flow (transshipment) — `min_cost_flow`

Send flow from supply nodes to demand nodes through capacitated arcs at least total cost (transshipment allowed): the minimum cost and a flow achieving it.

$$\min \sum c_{ij} x_{ij} \text{ s.t. } \sum_j x_{vj} - \sum_i x_{iv} = b_v,\ 0 \le x_{ij} \le u_{ij}$$

symbol		meaning	SI unit
`capacity`	in	arc capacities, n x n (row = from node, column = to node, 0 = no arc)	—
`cost`	in	unit cost of each arc, n x n	—
`supply`	in	net supply of each node (positive = supply, negative = demand), summing to 0	—
`min_cost`	out	minimum total cost	—
`flows`	out	optimal flow matrix (only when unique; otherwise flows_example)	—
`flow_unique`	out	yes when the optimal flow is unique	—
`dual`	out	dual values: node potentials then arc-capacity prices	—

Applies when: supplies sum to 0; a feasible flow exists; at most 60 arcs.

```
$ optimization solve min_cost_flow "capacity=[[0, 10, 5], [0, 0, 10], [0, 0, 0]]" "cost=[[0, 1, 4],
[0, 0, 2], [0, 0, 0]]" "supply=[8, 0, -8]"
min_cost = 24
flows = [[0.0,8.0,0.0],[0.0,0.0,8.0],[0.0,0.0,0.0]]
flow_unique = yes
dual = [3.0,2.0,0.0,0.0,0.0,0.0,0.0]
  by: Minimum-cost flow (transshipment)  (the min-cost-flow LP solved exactly (rational simplex)
and certified by its dual)
  assumes: supplies sum to 0; a feasible flow exists; at most 60 arcs
```

In the worked example, capacity = [0 10 5; 0 0 10; 0 0 0], cost = [0 1 4; 0 0 2; 0 0 0], supply = [8, 0, -8] go through the equation above and give min_cost = 24, flows = [[0, 8, 0], [0, 0, 8], [0, 0, 0]], flow_unique = yes, dual = [3, 2, 0, 0, 0, 0]. It applies only when supplies sum to 0; a feasible flow exists; at most 60 arcs: outside that it says what is wrong instead of returning a number.

Shortest path (Dijkstra's algorithm) — `shortest_path`

The length of the shortest route between two nodes of a graph with non-negative edge weights, and the route itself (stated as the route only when it is the only shortest one).

$$d(v) = \min_{(u,v)} \left(d(u) + w_{uv} \right)$$

symbol		meaning	SI unit
`edges`	in	weighted edges 'A-B 4, B-C 1, ...'	—
`source`	in	start node	—
`target`	in	end node	—
`directed`	in	true for a directed graph (edge u-v goes from u to v), false for undirected	—
`distance`	out	shortest distance	—
`path`	out	the shortest path (only when unique; otherwise path_example)	—
`path_unique`	out	yes when exactly one shortest path exists	—

Applies when: weights >= 0; a path from source to target exists; parallel edges: the lightest counts.

Run `optimization info shortest_path` for its inputs, then `optimization solve shortest_path name=value ...`.

What the worked example means: This calculation determines the minimum total weight required to travel from the starting node A to the destination node D using the provided edges. The result, 8, represents this optimal cost. If a single unique route exists, it is identified as [A, C, B, D]; otherwise, the system notes that multiple paths share this minimal cost.

Shortest path with negative weights (Bellman-Ford) — `bellman_ford_shortest_path`

Shortest distance in a directed graph whose edge weights may be negative, with all distances from the source; a negative cycle reachable from the source is reported instead of a number.

$$d(v) \leq d(u) + w_{uv} \; \forall (u, v)$$

symbol		meaning	SI unit
`edges`	in	directed weighted edges 'A-B 4, B-C -2, ...'	—
`source`	in	start node	—
`target`	in	end node	—
`distance`	out	shortest distance from source to target	—
`distances_from_source`	out	shortest distance to every reachable node	—

Applies when: directed graph; no negative cycle reachable from the source (otherwise INVALID_INPUT).

Run `optimization info bellman_ford_shortest_path` for its inputs, then `optimization solve bellman_ford_shortest_path name=value ...`.

What the worked example means: This calculation determines the minimum path cost between two nodes in a directed graph that may contain negative edge weights. By applying iterative relaxation, the algorithm finds the optimal route from S to T. The resulting value of 5 represents the sum of weights along this shortest path. If no such path exists or if a negative cycle is detected, the result would be invalid rather than a numerical distance.

Transportation problem (minimum shipping cost) — `transportation_problem`

Ship from sources with given supplies to destinations with given demands at least total cost: the minimum cost and a shipping plan (stated as THE plan only when it is unique).

$$\min \sum_{ij} c_{ij} x_{ij} \text{ s.t. } \sum_{j} x_{ij} \le s_i,\ \sum_{i} x_{ij} = d_j,\ x \ge 0$$

symbol		meaning	SI unit
`supply`	in	supply of each source	—
`demand`	in	demand of each destination	—
`cost`	in	unit shipping cost, rows = sources, columns = destinations	—
`min_cost`	out	minimum total shipping cost	—
`shipments`	out	optimal plan, rows = sources (only when unique; otherwise shipments_example)	—
`plan_unique`	out	yes when the optimal plan is unique	—
`dual`	out	dual values (u for sources, v for destinations): the optimality certificate	—

Applies when: total supply >= total demand (surplus stays at the sources at no cost); at most 60 routes.

```
$ optimization solve transportation_problem "supply=[20, 30]" "demand=[10, 25, 15]" "cost=[[2, 3,
1], [5, 4, 8]]"
min_cost = 150
shipments = [[5.0,0.0,15.0],[5.0,25.0,0.0]]
plan_unique = yes
dual = [-3.0,0.0,5.0,4.0,4.0]
  by: Transportation problem (minimum shipping cost)  (the transportation LP solved exactly
(rational simplex), certified by its dual (u_i + v_j <= c_ij))
  assumes: total supply >= total demand (surplus stays at the sources at no cost); at most 60
routes
```

In the worked example, supply = [20, 30], demand = [10, 25, 15], cost = [2 3 1; 5 4 8] go through the equation above and give min_cost = 150, shipments = [[5, 0, 15], [5, 25, 0]], plan_unique = yes, dual = [-3, 0, 5, 4, 4]. It applies only when total supply >= total demand (surplus stays at the sources at no cost); at most 60 routes: outside that it says what is wrong instead of returning a number.

Travelling salesman tour (exact, Held-Karp) — `traveling_salesman`

The shortest round trip visiting every city exactly once and returning to the start, for a distance matrix (symmetric or not): the minimum tour length and one optimal tour.

$$g(S, k) = \min_{m \in S \setminus \{k\}} \left(g(S \setminus \{k\}, m) + d_{mk} \right)$$

symbol		meaning	SI unit
`D`	in	distance matrix, D[i][j] = distance from city i to city j	—
`min_tour_length`	out	length of a shortest tour	—
`tour_example`	out	one optimal tour starting and ending at city 1	—

Applies when: 2 to 11 cities.

```
$ optimization solve traveling_salesman "D=[[0, 10, 15, 20], [10, 0, 35, 25], [15, 35, 0, 30], [20,
25, 30, 0]]"
min_tour_length = 80
tour_example = [1,3,4,2,1]
  by: Travelling salesman tour (exact, Held-Karp)  (Held-Karp dynamic programming over city
subsets, exact arithmetic)
  assumes: 2 to 11 cities
```

What the worked example means: This calculation determines the shortest possible route that visits every city exactly once and returns to the starting point, using dynamic programming over all subsets of cities. Given the distance matrix [0 10 15 20; 10 0 35 25; 15 35 0 30; 20 25 30 0], the algorithm computes the optimal path by minimizing the sum of segment distances across all permutations. The result is a specific sequence [1, 3, 4, 2, 1] with a total cost of 80, guaranteeing this is the absolute best solution rather than an approximation.

Nonlinear optimisation (3)

Equality-constrained quadratic program (Lagrange multipliers / KKT) — `equality_constrained_qp`

Minimise 1/2 x^T Q x + c^T x subject to A x = b: the minimiser, the minimum and the Lagrange multipliers.

$$\begin{bmatrix} Q & A^{\top} \\ A & 0 \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix} = \begin{bmatrix} -c \\ b \end{bmatrix}$$

symbol		meaning	SI unit
`Q`	in	symmetric positive definite Hessian Q	—
`c`	in	linear term c	—
`A`	in	constraint matrix A (rows = constraints)	—
`b`	in	right-hand side b	—
`f_min`	out	minimum value	—
`x_star`	out	the minimiser (unique)	—
`multipliers`	out	Lagrange multipliers lambda (convention Q x + c + A^T lambda = 0)	—

Applies when: Q symmetric positive definite; rows of A linearly independent.

```
$ optimization solve equality_constrained_qp "Q=2 0; 0 2" "c=0, 0" "A=1 1" b=2
f_min = 2
x_star = [1.0,1.0]
multipliers = [-2.0]
  by: Equality-constrained quadratic program (Lagrange multipliers / KKT)  (the KKT system [Q A^T;
A 0][x; lambda] = [-c; b] solved exactly; certified by stationarity and feasibility)
  assumes: Q symmetric positive definite; rows of A linearly independent
```

What the worked example means: This calculation determines the optimal solution for a quadratic minimization problem subject to linear equality constraints. Solving the associated system of linear equations derived from optimality conditions gives the specific decision variables that minimize the objective function. The result indicates that the lowest achievable cost or energy level is 2, occurring exactly when the variables take the values [1, 1]. Additionally, the sensitivity of this optimum to changes in the constraint limits is captured by the Lagrange multipliers, which are reported as [-2].

Steepest descent step with exact line search (quadratic) — `steepest_descent_step`

One iteration of steepest descent on f(x) = 1/2 x^T Q x + c^T x from x0: the gradient, the exact step length, the new point and its function value.

$$\alpha = \frac{g^\top g}{g^\top Q g}, \quad x_1 = x_0 - \alpha g$$

symbol		meaning	SI unit
`Q`	in	symmetric positive definite Hessian Q	—
`c`	in	linear term c	—
`x0`	in	starting point x0	—
`alpha`	out	exact step length g^T g / g^T Q g	—
`x1`	out	new point x0 - alpha g	—
`f1`	out	f at the new point	—
`gradient`	out	gradient at x0	—

Applies when: Q symmetric positive definite; x0 is not the minimiser.

```
$ optimization solve steepest_descent_step "Q=[[2, 0], [0, 8]]" "c=[0, 0]" "x0=[4, 1]"
alpha = 0.2
x1 = [2.4,-0.6]
f1 = 7.2
gradient = [8.0,8.0]
  by: Steepest descent step with exact line search (quadratic)  (g = Q x0 + c; alpha = g^T g / g^T
Q g in exact arithmetic; certified by g^T grad f(x1) = 0)
  assumes: Q symmetric positive definite; x0 is not the minimiser
```

What the worked example means: This calculation finds the optimal distance to move from the starting point in the direction opposite to the gradient. The gradient at the start is [8, 8], and the precise step length that minimizes the function along this line is 0.2. Applying this step yields the new position [2.4, -0.6], where the function value becomes 7.2. This ensures the next iteration begins with a significantly reduced error.

Unconstrained quadratic minimisation — `unconstrained_quadratic_min`

Minimise f(x) = 1/2 x^T Q x + c^T x for a symmetric positive definite Q: the minimiser and the minimum.

$$x^* = -Q^{-1}c, \quad f^* = \tfrac{1}{2}c^\top x^*$$

symbol		meaning	SI unit
`Q`	in	symmetric positive definite Hessian Q (n x n)	—
`c`	in	linear term c (n entries)	—
`f_min`	out	minimum value	—
`x_star`	out	the minimiser x* = -Q^{-1} c (unique)	—

Applies when: Q symmetric positive definite (convention: f = 1/2 x^T Q x + c^T x); n <= 10.

```
$ optimization solve unconstrained_quadratic_min "Q=[[4, 1], [1, 3]]" "c=[-1, -2]"
f_min = -0.6818181818
x_star = [0.09090909090909091,0.6363636363636364]
  by: Unconstrained quadratic minimisation  (positive definiteness by symmetric elimination, then Q
x = -c solved exactly; certified by the gradient)
  assumes: Q symmetric positive definite (convention: f = 1/2 x^T Q x + c^T x); n <= 10
```

What the worked example means: This calculation determines the global minimum of a quadratic function defined by a symmetric positive definite matrix. It solves for the point where the gradient vanishes, yielding the optimal solution vector [0.0909091, 0.636364] and the corresponding objective value -0.681818. The result represents the lowest possible cost achievable under these conditions.

Project scheduling (2)

Critical path method (project duration and slack) — `critical_path_method`

For activities with durations and precedences, the minimum project duration, the critical activities and each activity's earliest start, latest start and slack (total float).

$$ES_k = \max_{j \to k}(ES_j + d_j), \quad LS_k = \min_{k \to s} LS_s - d_k, \quad \text{slack}_k = LS_k - ES_k$$

symbol		meaning	SI unit
`durations`	in	duration of each activity; activities are named A, B, C, ... (or 1, 2, 3, ...) in this order	—
`precedences`	in	precedence pairs 'A-B' meaning A must finish before B starts (none if omitted) (optional)	—
`project_duration`	out	minimum project duration	—
`critical_activities`	out	activities with zero slack	—
`slack`	out	total slack of each activity	—
`earliest_start`	out	earliest start of each activity	—
`latest_start`	out	latest start of each activity	—

Applies when: finish-to-start precedences without cycles; deterministic durations.

```
$ optimization solve critical_path_method "durations=[3, 4, 2, 5]" "precedences=A-B, A-C, B-D, C-D"
project_duration = 12
critical_activities = ["A","B","D"]
slack = [0.0,0.0,2.0,0.0]
earliest_start = [0.0,3.0,3.0,7.0]
latest_start = [0.0,3.0,5.0,7.0]
  by: Critical path method (project duration and slack)  (forward pass (earliest starts) and
backward pass (latest finishes) on the activity-on-node network, exact fractions)
  assumes: finish-to-start precedences without cycles; deterministic durations
```

What the worked example means: The optimization model computes the earliest start, latest start, and total slack for each activity based on their durations and precedence constraints. The resulting minimum project duration of 12 represents the longest path through the network, meaning any delay in the critical activities [A,

B, D] will directly extend the project timeline. Non-critical tasks possess positive slack values, indicating how much they can be delayed without affecting the overall completion date.

PERT project duration and completion probability — `pert_project`

With optimistic, most likely and pessimistic times per activity, the expected project duration, its variance along the critical path and the probability of finishing by a deadline (normal approximation).

$$t_e = \frac{a + 4m + b}{6},\ \sigma^2 = \left(\frac{b - a}{6}\right)^2,\ P(T \le d) = \Phi\left(\frac{d - \mu}{\sigma}\right)$$

symbol		meaning	SI unit
`times`	in	rows (optimistic, most likely, pessimistic), one per activity A, B, C, ...	—
`precedences`	in	precedence pairs 'A-B' (A before B) (optional)	—
`deadline`	in	deadline for the completion probability (optional) (optional)	—
`expected_duration`	out	expected project duration	—
`variance`	out	variance of the critical path	—
`std_dev`	out	standard deviation	—
`critical_path`	out	the critical path (must be unique)	—
`expected_times`	out	expected time of each activity (a + 4m + b)/6	—
`probability`	out	probability of completion by the deadline	—

Applies when: a unique critical path (otherwise INVALID_INPUT); independent activity times; normal approximation.

```
$ optimization solve pert_project "times=[[2, 4, 6], [3, 5, 13], [1, 2, 3]]" "precedences=A-B, A-C"
expected_duration = 10
variance = 3.222222222
std_dev = 1.795054936
critical_path = ["A","B"]
expected_times = [4.0,6.0,2.0]
  by: PERT project duration and completion probability  (beta-PERT expected times, longest path in
exact fractions with a count of longest paths, variance summed along it)
  assumes: a unique critical path (otherwise INVALID_INPUT); independent activity times; normal
approximation
```

What the worked example means: This calculation determines the expected project duration and its variability based on optimistic, most likely, and pessimistic time estimates for each activity. Applying beta-PERT formulas gives an expected time for every task and identifies the unique critical path, which dictates the overall schedule risk. The result shows that the project has an expected duration of 10 and a standard deviation of 1.79505, indicating the spread of possible completion times around this mean.

Queueing theory (7)

Erlang B blocking probability (M/M/c/c loss system) — `erlang_b`

The probability that a call (customer) is blocked in a loss system with c servers and no waiting room, for an offered traffic of A erlangs, with the carried and lost traffic.

$$B(c, A) = \frac{A^c/c!}{\sum_{k=0}^{c} A^k/k!}$$

symbol		meaning	SI unit
`A`	in	offered traffic in erlangs (arrival rate x mean holding time)	—
`c`	in	number of servers (lines, trunks)	—
`blocking_probability`	out	Erlang B blocking probability	—
`carried_traffic`	out	carried traffic A (1 - B), erlangs	—
`lost_traffic`	out	lost traffic A B, erlangs	—

Applies when: blocked customers are lost (no waiting); A > 0.

```
$ optimization solve erlang_b A=3 c=5
blocking_probability = 0.1100543478
carried_traffic = 2.669836957
lost_traffic = 0.3301630435
  by: Erlang B blocking probability (M/M/c/c loss system)  (stable recursion B(k) = A B(k-1) / (k +
A B(k-1)))
  A = 3
  assumes: blocked customers are lost (no waiting); A > 0
```

What the worked example means: This calculation determines the likelihood that a call is blocked in a loss system with no waiting room. Given an offered traffic of 3 erlangs and 5 servers, the model computes a blocking probability of 0.110054. Consequently, the system successfully carries 2.66984 erlangs of traffic, while 0.330163 erlangs are lost due to insufficient capacity.

M/G/1 queue (Pollaczek-Khinchine formula) — `mg1_pollaczek_khinchine`

Mean queue length and waiting times of a single-server queue with Poisson arrivals and a GENERAL service time distribution given by its mean and standard deviation (M/D/1 when the standard deviation is 0).

$$L_q = \frac{\lambda^2 \sigma_S^2 + \rho^2}{2(1-\rho)}$$

symbol		meaning	SI unit
`lam`	in	arrival rate	—
`ES`	in	mean service time (time unit of 1/lambda)	—
`sd`	in	standard deviation of the service time	—
`rho`	out	utilisation lambda E[S]	—
`Lq`	out	mean number waiting	—
`L`	out	mean number in the system	—
`Wq`	out	mean waiting time	—
`W`	out	mean time in the system	—

Applies when: lambda E[S] < 1; Poisson arrivals, one server, first come first served.

```
$ optimization solve mg1_pollaczek_khinchine lam=0.5 ES=1 sd=1
rho = 0.5
Lq = 0.5
L = 1
Wq = 1
W = 2
  by: M/G/1 queue (Pollaczek-Khinchine formula)  (Pollaczek-Khinchine mean-value formula; Little's
law)
  lam = 0.5
  ES = 1
  sd = 1
  assumes: lambda E[S] < 1; Poisson arrivals, one server, first come first served
```

What the worked example means: This calculation determines the average queue length and waiting times for a single server handling Poisson arrivals with general service times. Using the arrival rate of 0.5 and service characteristics defined by a mean time of 1 and standard deviation of 1, the system yields a utilisation of 0.5. The resulting mean number of customers waiting is 0.5, while the total mean number in the system is 1. Consequently, the average time spent waiting in line is 1, and the total time in the system is 2.

M/M/1 queue — `mm1_queue`

Steady-state measures of a single-server queue with Poisson arrivals and exponential service: utilisation, mean numbers in system and in queue, mean times in system and in queue, probability of an empty system.

$$L = \frac{\rho}{1-\rho},\ W = \frac{1}{\mu-\lambda},\ W_q = \frac{\rho}{\mu-\lambda}$$

symbol		meaning	SI unit
`lam`	in	arrival rate (customers per unit time)	—
`mu`	in	service rate (customers per unit time)	—
`rho`	out	utilisation lambda/mu	—
`L`	out	mean number in the system	—
`Lq`	out	mean number waiting	—
`W`	out	mean time in the system (time unit of the rates)	—
`Wq`	out	mean waiting time in the queue	—
`P0`	out	probability the system is empty	—

Applies when: lambda < mu (stable); Poisson arrivals, exponential service, one server, unlimited capacity.

```
$ optimization solve mm1_queue lam=8 mu=10
rho = 0.8
L = 4
Lq = 3.2
W = 0.5
Wq = 0.4
P0 = 0.2
  by: M/M/1 queue  (geometric stationary distribution; Little's law)
  lam = 8
  mu = 10
  assumes: lambda < mu (stable); Poisson arrivals, exponential service, one server, unlimited
capacity
```

What the worked example means: This calculation determines steady-state performance for a single-server queue. Given an arrival rate of 8 and a service rate of 10, the system utilisation is computed as 0.8. Consequently, the average number of customers in the system is 4, while those waiting in the queue average 3.2. The mean time spent in the entire system is 0.5, and the average wait before service begins is 0.4. Finally, the probability that the system is completely empty is 0.2.

M/M/c queue (Erlang C) — `mmc_queue`

Steady-state measures of a queue with c parallel identical servers, Poisson arrivals and exponential service: probability of waiting (Erlang C), mean queue length and waiting times.

$$C(c,a) = \frac{B(c,a)}{1-\rho(1-B(c,a))}, \quad L_q = C\,\frac{\rho}{1-\rho}$$

symbol		meaning	SI unit
`lam`	in	arrival rate	—
`mu`	in	service rate of each server	—
`c`	in	number of servers	—
`rho`	out	utilisation lambda/(c mu)	—
`P_wait`	out	probability an arrival waits (Erlang C)	—
`Lq`	out	mean number waiting	—
`Wq`	out	mean waiting time	—
`W`	out	mean time in the system	—
`L`	out	mean number in the system	—
`P0`	out	probability the system is empty	—

Applies when: lambda < c mu (stable); unlimited queue capacity.

```
$ optimization solve mmc_queue lam=24 mu=10 c=3
rho = 0.8
P_wait = 0.6471910112
Lq = 2.588764045
Wq = 0.1078651685
W = 0.2078651685
L = 4.988764045
P0 = 0.05617977528
  by: M/M/c queue (Erlang C)  (Erlang B by its stable recursion, then Erlang C = B / (1 - rho (1 -
B)); Little's law)
  lam = 24
  mu = 10
  assumes: lambda < c mu (stable); unlimited queue capacity
```

What the worked example means: The system computes steady-state queue metrics for a multi-server setup. With an arrival rate of 24 per time unit and individual server capacity of 10, the total utilization is 0.8. The calculated probability that an arriving customer must wait in line is 0.647191. Consequently, the mean number of customers waiting in the queue is 2.58876, and the average time spent waiting before service begins is 0.107865. These results reflect the balance between demand and available parallel processing capacity.

M/M/c/K queue (finite capacity) — `mmck_finite_queue`

Steady-state measures of a queue with c servers and room for at most K customers in the system (arrivals finding it full are lost): blocking probability, effective arrival rate, mean numbers and times.

$$\pi_n \propto \frac{a^n}{n!}\ (n \le c), \quad \frac{a^c}{c!}\left(\frac{a}{c}\right)^{n-c}\ (c < n \le K)$$

symbol		meaning	SI unit
`lam`	in	arrival rate	—
`mu`	in	service rate of each server	—
`c`	in	number of servers	—
`K`	in	system capacity (customers in service plus waiting)	—
`P_block`	out	probability an arrival is lost (system full)	—
`lam_eff`	out	effective arrival rate	—
`L`	out	mean number in the system	—
`Lq`	out	mean number waiting	—
`W`	out	mean time in the system	—
`Wq`	out	mean waiting time	—
`P0`	out	probability the system is empty	—

Applies when: 1 <= c <= K; any utilisation (finite capacity is always stable).

```
$ optimization solve mmck_finite_queue lam=2 mu=2 c=1 K=3
P_block = 0.25
lam_eff = 1.5
L = 1.5
Lq = 0.75
W = 1
Wq = 0.5
P0 = 0.25
  by: M/M/c/K queue (finite capacity)  (truncated birth-death distribution computed in log space;
Little's law with the effective arrival rate)
  lam = 2
  mu = 2
  assumes: 1 <= c <= K; any utilisation (finite capacity is always stable)
```

What the worked example means: This calculation determines steady-state performance for a finite-capacity queue. Given an arrival rate of 2, a service rate of 2, 1 servers, and a maximum capacity of 3, the model computes the probability that an arriving customer finds the system full and is lost, which is 0.25. The effective arrival rate entering the system is 1.5, leading to an average number of customers in the system of 1.5 and an average waiting time of 0.5.

Machine-repair (finite-source) queue — `machine_repair_queue`

N machines each fail at rate lambda while running and c repairers fix them at rate mu each: mean number of machines down and waiting, failure rate, downtime and the mean number working.

$$\pi_n \propto \frac{N!}{(N-n)!}\,\frac{(\lambda/\mu)^n}{n!\text{ or }c!\,c^{n-c}}$$

symbol		meaning	SI unit
`N`	in	number of machines	—
`lam`	in	failure rate of one running machine	—
`mu`	in	repair rate of one repairer	—
`c`	in	number of repairers (default 1) (optional)	—
`L`	out	mean number of machines down	—
`Lq`	out	mean number waiting for repair	—
`lam_eff`	out	mean failure rate lambda (N - L)	—
`W`	out	mean downtime per failure	—
`Wq`	out	mean wait for a repairer	—
`P0`	out	probability all machines run	—
`working`	out	mean number running	—

Applies when: 1 <= c <= N; exponential failure and repair times.

```
$ optimization solve machine_repair_queue N=5 lam=0.2 mu=1 c=1
L = 1.424339107
Lq = 0.709206928
lam_eff = 0.7151321787
W = 1.991714468
Wq = 0.9917144678
P0 = 0.2848678213
working = 3.575660893
  by: Machine-repair (finite-source) queue  (finite birth-death distribution (birth (N - n) lambda,
death min(n, c) mu) in log space; Little's law)
  lam = 0.2
  mu = 1
  assumes: 1 <= c <= N; exponential failure and repair times
```

What the worked example means: This calculation determines the steady-state performance of a finite-source queueing system where 5 machines operate with individual failure rates of 0.2 and are serviced by 1 repairers at a rate of 1. The primary output, 1.42434, represents the average number of machines that are currently broken or waiting for repair. This gives the mean downtime per failure, 1.99171, and the probability that all machines are operational, 0.284868. These metrics allow operators to assess whether the current staffing level of 1 repairers is sufficient to maintain production targets given the observed failure behavior.

Minimum number of servers for a service target (Erlang C) — `mmc_staffing`

The smallest number of servers c of an M/M/c queue for which the probability of waiting, or the mean waiting time, meets a target.

$$c^* = \min\{c > \lambda/\mu : C(c, \lambda/\mu) \le \alpha\}$$

symbol		meaning	SI unit
`lam`	in	arrival rate	—
`mu`	in	service rate of each server	—
`target`	in	target value (a probability, or a mean wait in the time unit of the rates)	—
`criterion`	in	wait_probability (default) or mean_wait (optional)	—
`servers`	out	minimum number of servers	—
`P_wait`	out	probability of waiting with that many servers	—
`Wq`	out	mean waiting time with that many servers	—
`utilisation`	out	server utilisation	—

Applies when: Poisson arrivals, exponential service; target > 0.

Run `optimization info mmc_staffing` for its inputs, then `optimization solve mmc_staffing name=value ...`.

What the worked example means: This calculation determines the smallest number of servers needed for an M/M/c queue to satisfy the specified service level. By incrementally increasing the server count from the traffic intensity floor, the algorithm finds that 5 servers are sufficient. With this configuration, the probability that an arriving customer must wait is 0.113504, and the mean waiting time is 0.00436553. The resulting server utilisation is 0.48, indicating how busy the system is under these conditions.

Revenue management (1)

Littlewood's rule (two-fare protection level) — `littlewood_protection`

How many seats (units) to protect for full-fare demand when a discount fare is also sold: the protection level and, with a capacity, the booking limit for the discount class.

$$P(D_1 > y^*) = \frac{p_2}{p_1}, \quad y^* = \mu + z\,\sigma$$

symbol		meaning	SI unit
`p1`	in	full fare	—
`p2`	in	discount fare	—
`mu`	in	mean full-fare demand	—
`sigma`	in	standard deviation of full-fare demand	—
`capacity`	in	total capacity (optional) (optional)	—
`protection_level`	out	units protected for full-fare demand	—
`z`	out	standard normal quantile of 1 - p2/p1	—
`booking_limit`	out	discount booking limit = capacity - protection (when a capacity is given)	—

Applies when: 0 < p2 < p1; normally distributed full-fare demand; continuous approximation (no rounding).

```
$ optimization solve littlewood_protection p1=400 p2=150 mu=60 sigma=20
protection_level = 66.37278728
z = 0.318639364
  by: Littlewood's rule (two-fare protection level)  (protect y with P(D1 > y) = p2/p1)
  p1 = 400
  p2 = 150
  mu = 60
  sigma = 20
  assumes: 0 < p2 < p1; normally distributed full-fare demand; continuous approximation (no
rounding)
```

What the worked example means: This calculation determines the number of units to reserve for high-value customers, known as the protection level. Comparing the discount fare 150 to the full fare 400 establishes a critical fractile that guides how much demand to protect against early sell-outs. The resulting protection level is 66.3728, which balances the risk of turning away full-fare buyers against the opportunity cost of leaving seats empty or selling them at a lower price.

Scheduling (6)

Earliest due date rule (minimum maximum lateness) — `edd_max_lateness`

The job order that minimises the maximum lateness (and so the maximum tardiness) on one machine.

$$\min L_{\max} = \min \max_j (C_j - d_j) : \; d_{[1]} \le d_{[2]} \le \dots$$

symbol		meaning	SI unit
`p`	in	processing time of each job	—
`d`	in	due date of each job	—
`max_lateness`	out	minimum possible maximum lateness	—
`max_tardiness`	out	minimum possible maximum tardiness	—
`sequence`	out	optimal order (earliest due date first; ties in job-number order)	—
`tardy_jobs_in_sequence`	out	number of late jobs in this sequence (not minimised)	—

Applies when: all jobs available at time 0.

```
$ optimization solve edd_max_lateness "p=[2, 4, 1, 3]" "d=[5, 6, 3, 9]"
sequence = [3,1,2,4]
max_lateness = 1
max_tardiness = 1
tardy_jobs_in_sequence = 2
  by: Earliest due date rule (minimum maximum lateness)  (sort by due date (Jackson's rule))
  assumes: all jobs available at time 0
```

What the worked example means: By sorting jobs by due date, the algorithm determines the sequence [3, 1, 2, 4] that minimizes the worst-case delay. This results in a maximum lateness of 1 and a maximum tardiness of 1, with 2 jobs finishing late.

Johnson's rule (two-machine flow shop) — `johnson_two_machine`

The job order that minimises the makespan when every job is processed first on machine 1 and then on machine 2.

$$\min C_{\max} \text{ for } F2 \| C_{\max}$$

symbol		meaning	SI unit
`a`	in	processing time of each job on machine 1	—
`b`	in	processing time of each job on machine 2	—
`makespan`	out	minimum makespan	—
`sequence`	out	Johnson order (a < b by increasing a, then the rest by decreasing b; ties by job number)	—

Applies when: same job order on both machines; all jobs available at time 0.

```
$ optimization solve johnson_two_machine "a=[5, 2, 6, 7]" "b=[3, 4, 8, 2]"
makespan = 22
sequence = [2,3,1,4]
  by: Johnson's rule (two-machine flow shop)  (Johnson's rule)
  assumes: same job order on both machines; all jobs available at time 0
```

What the worked example means: Johnson's rule determines the best order for jobs in a two-machine flow shop to minimize total completion time. By comparing processing times on machine one and machine two, it assigns jobs with shorter first-stage times early and those with shorter second-stage times late. The resulting

schedule achieves a makespan of 22, ensuring efficient resource utilization under the assumption that all jobs are available at the start and follow the same routing path.

Minimum makespan on identical parallel machines — `parallel_machine_makespan`

Assign jobs to m identical machines so that the last machine finishes as early as possible (P||Cmax), exactly.

$$\min_{\text{assignment}} \max_{k} \sum_{j \to k} p_j$$

symbol		meaning	SI unit
`p`	in	processing time of each job	—
`m`	in	number of identical machines	—
`makespan`	out	minimum makespan	—
`machine_of_job`	out	one optimal assignment: machine number of each job	—
`lower_bound`	out	the bound max(longest job, total / m)	—

Applies when: at most 12 jobs and 6 machines; jobs are not split.

```
$ optimization solve parallel_machine_makespan "p=[7, 5, 4, 3, 3, 2]" m=2
makespan = 12
machine_of_job = [1,2,2,1,2,1]
lower_bound = 12
  by: Minimum makespan on identical parallel machines  (exact depth-first branch and bound (longest
job first, symmetric machines skipped), started from the LPT schedule)
  assumes: at most 12 jobs and 6 machines; jobs are not split
```

What the worked example means: This calculation determines the optimal schedule for identical parallel machines using an exact branch and bound algorithm. Given processing times of [7, 5, 4, 3, 3, 2] on 2 machines, it finds the assignment that minimizes the maximum completion time across all machines. The resulting minimum makespan is 12, with the lower bound estimated at 12. This ensures no other valid assignment can finish earlier than this value.

Moore-Hodgson algorithm (minimum number of tardy jobs) — `moore_hodgson_tardy_jobs`

The smallest possible number of late jobs on one machine, with an order of the on-time jobs and the set of late jobs.

$$\min \sum_{j} U_j$$

symbol		meaning	SI unit
`p`	in	processing time of each job	—
`d`	in	due date of each job	—
`min_tardy_jobs`	out	minimum number of tardy jobs	—
`on_time_sequence`	out	on-time jobs in due-date order	—

symbol		meaning	SI unit
`tardy_jobs`	out	jobs scheduled late (one optimal choice: the longest job removed, ties the higher job number)	—

Applies when: all jobs available at time 0.

```
$ optimization solve moore_hodgson_tardy_jobs "p=[4, 3, 2, 5, 6]" "d=[5, 6, 8, 12, 15]"
min_tardy_jobs = 2
on_time_sequence = [2,3,4]
tardy_jobs = [1,5]
  by: Moore-Hodgson algorithm (minimum number of tardy jobs)  (EDD order; whenever a job is late
remove the longest job scheduled so far)
  assumes: all jobs available at time 0
```

What the worked example means: This calculation determines the smallest possible count of late jobs using the Moore-Hodgson algorithm. Starting from an Earliest Due Date order, it removes the job with the longest processing time whenever a lateness occurs. The result shows that only 2 jobs must be delayed, while the remaining tasks in the sequence [2, 3, 4] finish on time.

Shortest processing time rule (minimum mean flow time) — `spt_mean_flow_time`

The job order that minimises the mean flow (completion) time on one machine, and that minimum.

$$\min \frac{1}{n} \sum_{j} C_j : \; p_{[1]} \leq p_{[2]} \leq \dots$$

symbol		meaning	SI unit
`p`	in	processing time of each job (jobs numbered 1, 2, ...)	—
`mean_flow_time`	out	minimum mean completion time	—
`total_completion_time`	out	minimum total completion time	—
`sequence`	out	optimal job order (shortest first; equal times in job-number order)	—

Applies when: all jobs available at time 0; processing times >= 0.

```
$ optimization solve spt_mean_flow_time "p=[4, 2, 6, 3]"
sequence = [2,4,1,3]
total_completion_time = 31
mean_flow_time = 7.75
  by: Shortest processing time rule (minimum mean flow time)  (sort by processing time (SPT);
optimal by the exchange argument)
  assumes: all jobs available at time 0; processing times >= 0
```

What the worked example means: This calculation determines the job order that minimizes the average completion time on a single machine. Sorting jobs by processing time from shortest to longest gives the sequence [2, 4, 1, 3]. This arrangement yields a total completion time of 31 and a minimum mean flow time of 7.75, ensuring the most efficient use of the machine for this specific objective.

Weighted shortest processing time (Smith's rule) — `wspt_weighted_completion`

The job order that minimises the total weighted completion time on one machine, and that minimum.

$$\min \sum_j w_j C_j : \ \frac{p_{[1]}}{w_{[1]}} \le \frac{p_{[2]}}{w_{[2]}} \le \dots$$

symbol		meaning	SI unit
`p`	in	processing time of each job	—
`w`	in	weight (importance) of each job	—
`weighted_completion_time`	out	minimum total weighted completion time	—
`sequence`	out	optimal order (ratio p/w ascending; ties in job-number order)	—

Applies when: weights > 0; all jobs available at time 0.

```
$ optimization solve wspt_weighted_completion "p=[3, 1, 4]" "w=[1, 2, 3]"
sequence = [2,3,1]
weighted_completion_time = 25
  by: Weighted shortest processing time (Smith's rule)  (sort by p_j / w_j (Smith's ratio rule))
  assumes: weights > 0; all jobs available at time 0
```

What the worked example means: This calculation determines the job order that minimizes the sum of each job's weight multiplied by its completion time. By sorting jobs based on the ratio of processing time to weight, the algorithm identifies the most efficient schedule. The resulting sequence is [2, 3, 1], which achieves a total weighted completion time of 25. This means that high-weight jobs with short processing times are prioritized to reduce overall cost.

Unconstrained optimisation (1)

Optimum of a quadratic function — `quadratic_optimum`

The minimum or maximum of f(x) = a x^2 + b x + c: where it occurs and its value.

$$x^* = -\frac{b}{2a}, \quad f^* = c - \frac{b^2}{4a}$$

symbol		meaning	SI unit
`a`	in	coefficient of x^2	—
`b`	in	coefficient of x	—
`c`	in	constant term	—
`goal`	in	minimum, maximum or optimum (default optimum) (optional)	—
`x_opt`	out	x at the optimum	—
`f_opt`	out	optimal value f(x*)	—
`type`	out	minimum (a > 0) or maximum (a < 0)	—

Applies when: a != 0; a minimum asked needs a > 0, a maximum a < 0.

```
$ optimization solve quadratic_optimum a=2 b=-8 c=3 goal=minimum
x_opt = 2
f_opt = -5
type = minimum
  by: Optimum of a quadratic function  (first-order condition f'(x) = 2ax + b = 0, exact fractions)
  a = 2
  b = -8
  c = 3
  goal = minimum
  assumes: a != 0; a minimum asked needs a > 0, a maximum a < 0
```

What the worked example means: This calculation determines the vertex of the parabolic curve defined by the coefficients a, b, and c. Applying the first-order condition shows that the extremum is located at x equals 2. The corresponding optimal value of the function is computed as -5. Depending on the sign of the leading coefficient, this point represents either a minimum or a maximum.

Questions and reports: support@ai999x.com. AI999X — The Future Is Self-Improving.